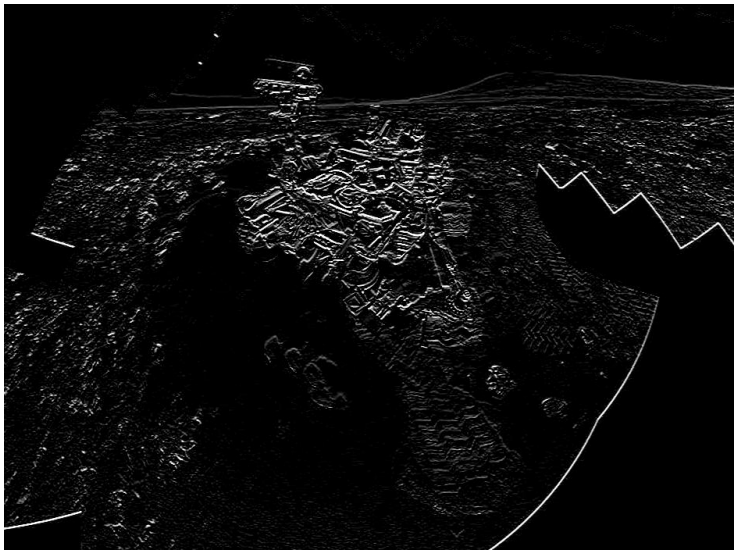
Spatial Filtering

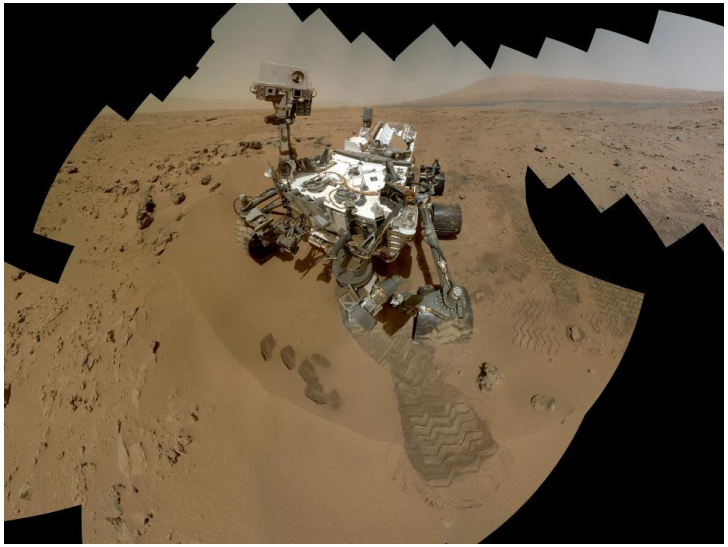
CS 4330 Mathias Kölsch

January 2025

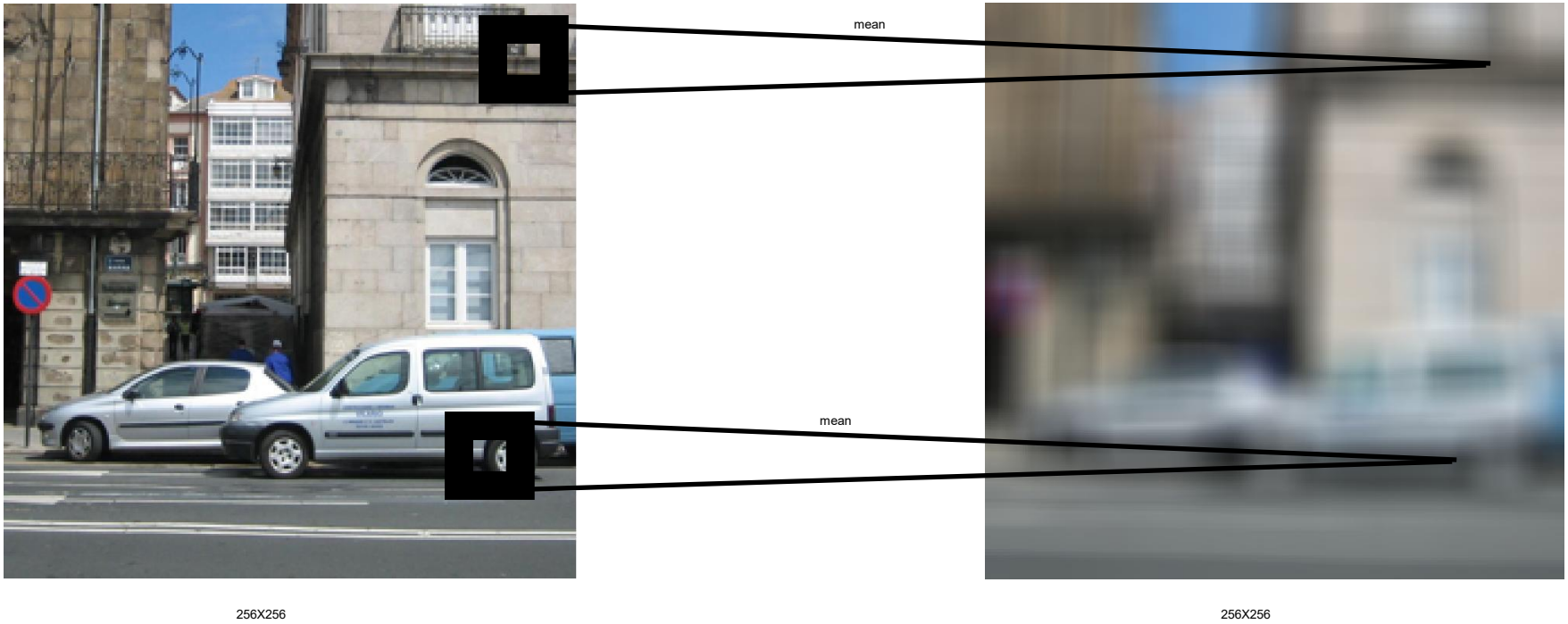
Today

- correlation, convolution
- box filters
- averaging, sharpening
- Gaussian
- 1st and 2nd derivatives
- Laplacian





Box filter



What does it do?

- Replaces each pixel with an **average** of its neighborhood
- Achieves smoothing effect (removes sharp, high-contrast features)

Correlation

```
                Origin      f                w
(a) 0 0 0 1 0 0 0 0         1 2 3 2 0

                  ↓
(b)               0 0 0 1 0 0 0 0
    1 2 3 2 0
                  ↑ Starting position alignment

                 Zero padding
(c) 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
    1 2 3 2 0

(d) 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
      1 2 3 2 0
      ↑ Position after one shift

(e) 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
                1 2 3 2 0
                ↑ Position after four shifts

(f) 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
                            1 2 3 2 0
                Final position ↑

            'full' correlation result
(g)       0 0 0 0 2 3 2 1 0 0 0 0
```

from: Gonzalez, Woods, Eddins
Digital Image Processing using Matlab

Origin of $f(x, y)$

0	0	0	0	0
0	0	0	0	0
0	0	1	0	0
0	0	0	0	0
0	0	0	0	0

$w(x, y)$

1	2	3
4	5	6
7	8	9

(a)

Padded f

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

(b)

Initial position for w

1	2	3	0	0	0	0	0	0
4	5	6	0	0	0	0	0	0
7	8	9	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

(c)

`'full'` correlation result

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	9	8	7	0	0	0
0	0	0	6	5	4	0	0	0
0	0	0	3	2	1	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

(d)

Rotated w

9	8	7	0	0	0	0	0	0
6	5	4	0	0	0	0	0	0
3	2	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

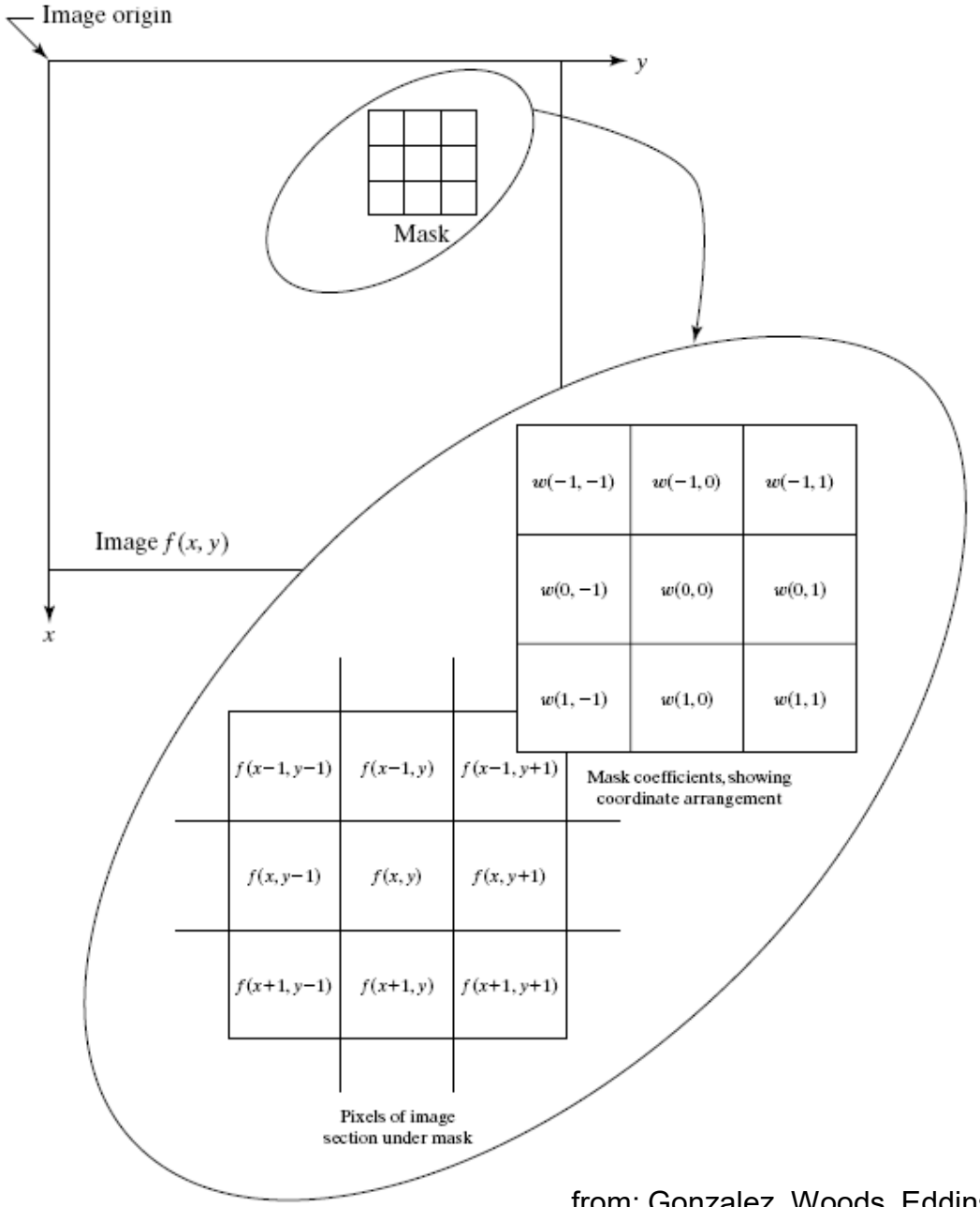
(f)

`'full'` convolution result

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	1	2	3	0	0	0
0	0	0	4	5	6	0	0	0
0	0	0	7	8	9	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

(g)

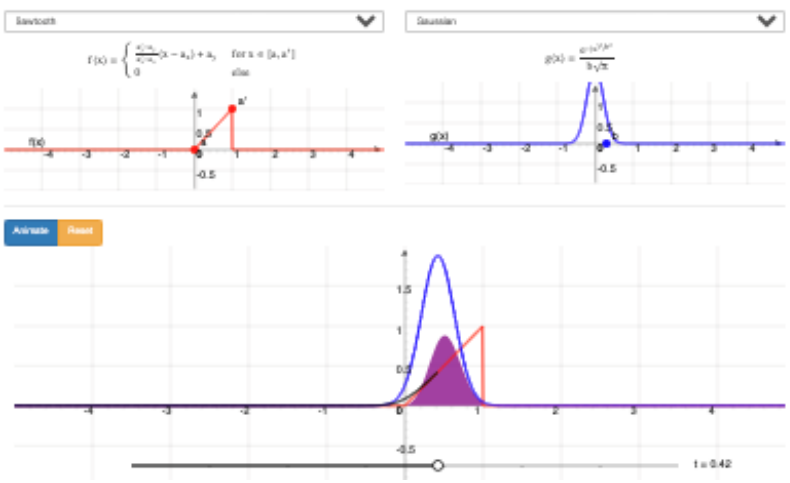
from: Gonzalez, Woods, Eddins
Digital Image Processing using Matlab

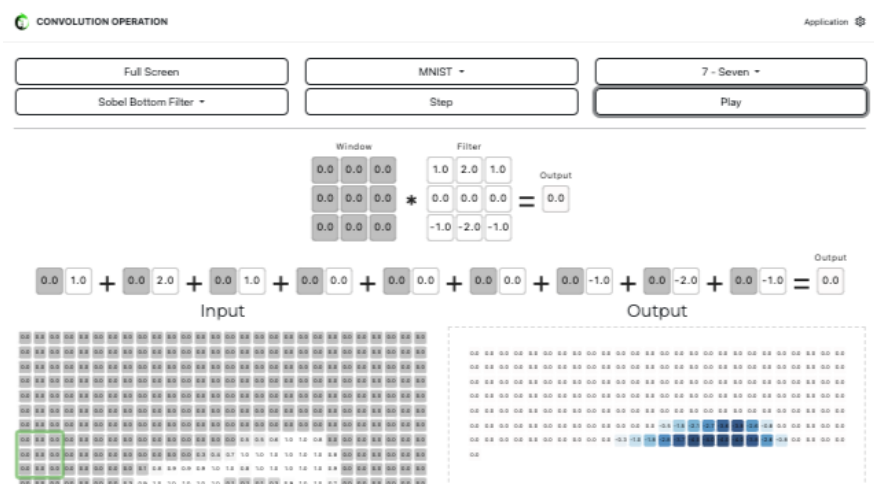


from: Gonzalez, Woods, Eddins
Digital Image Processing using Matlab

Resources

- Lots of material on the web, for example
- 1D https://phiresky.github.io/convolution-demo/
- 2D https://deeplizard.com/resource/pavq7noze2





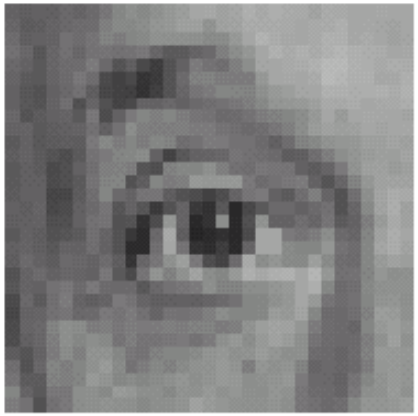
Spatial Filtering

$$R_{ij} = \sum_{u,v} H_{i-u,j-v} F_{uv}$$

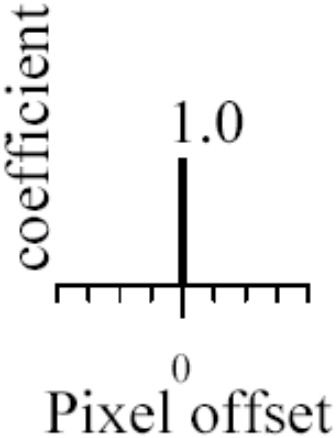
$$f[m,n] = I \otimes g = \sum_{k,l} I[m-k, n-l] g[k,l]$$

note the negative direction

Linear filtering (warm-up slide)

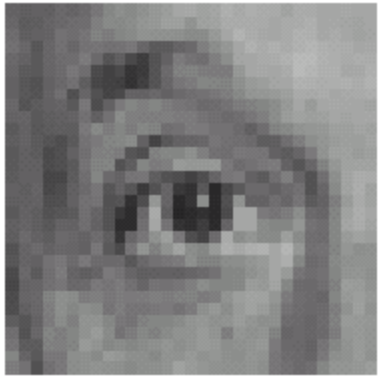


original

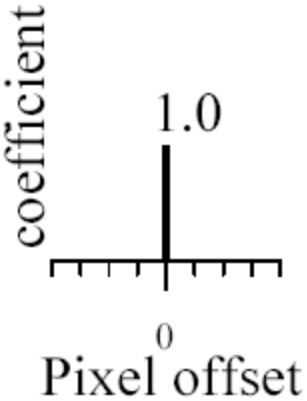


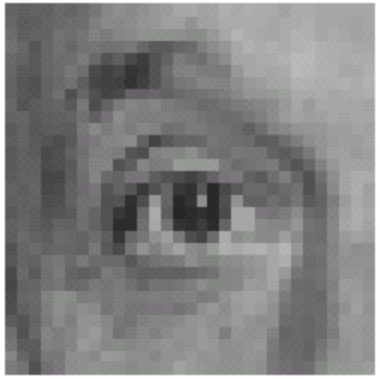
?

Linear filtering (warm-up slide)



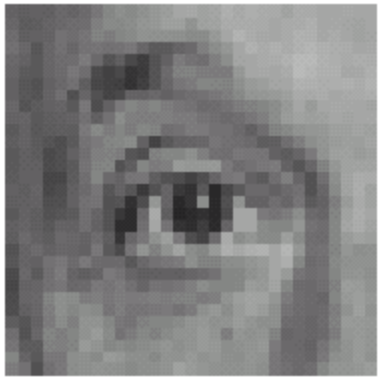
original



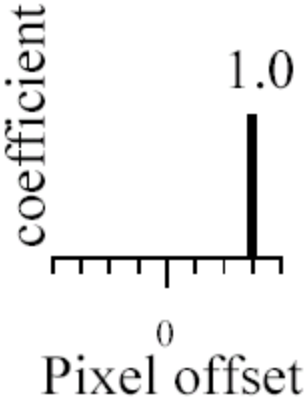


Filtered
(no change)

Linear filtering

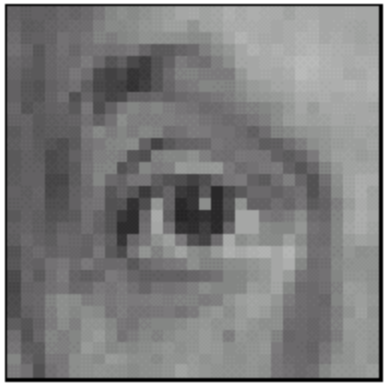


original

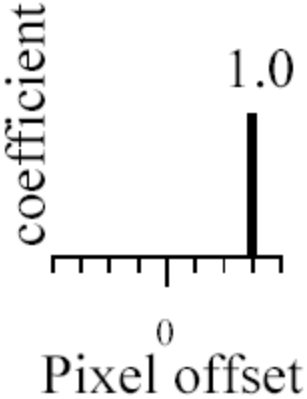


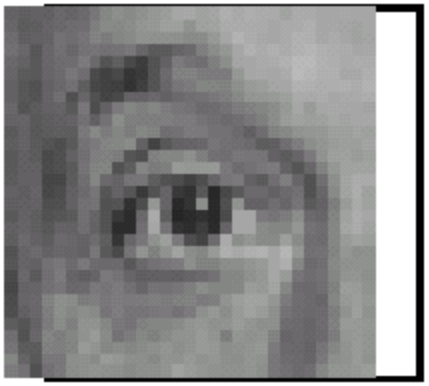
?

shift



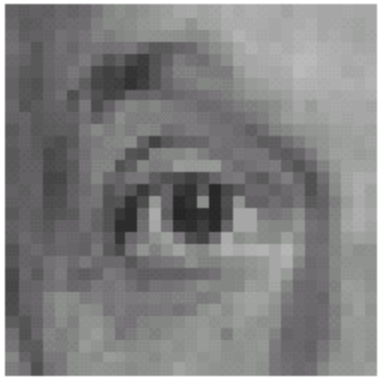
original



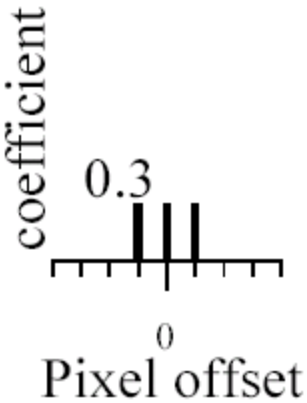


shifted

Linear filtering

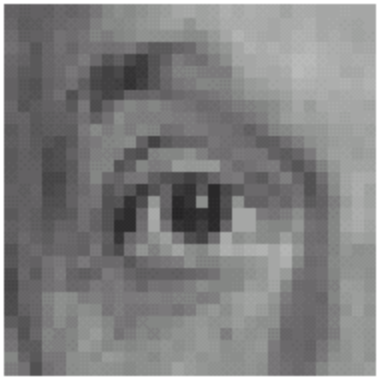


original

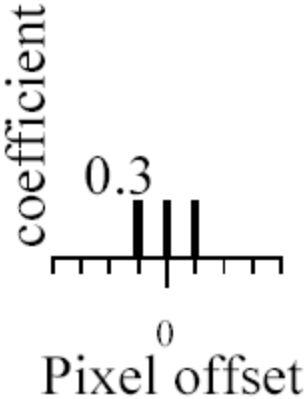


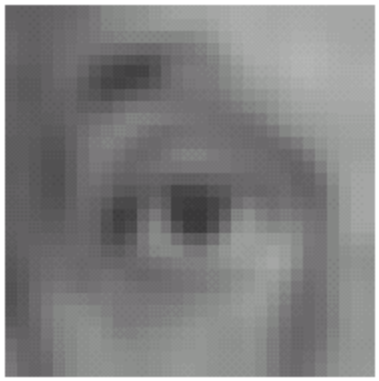
?

Blurring



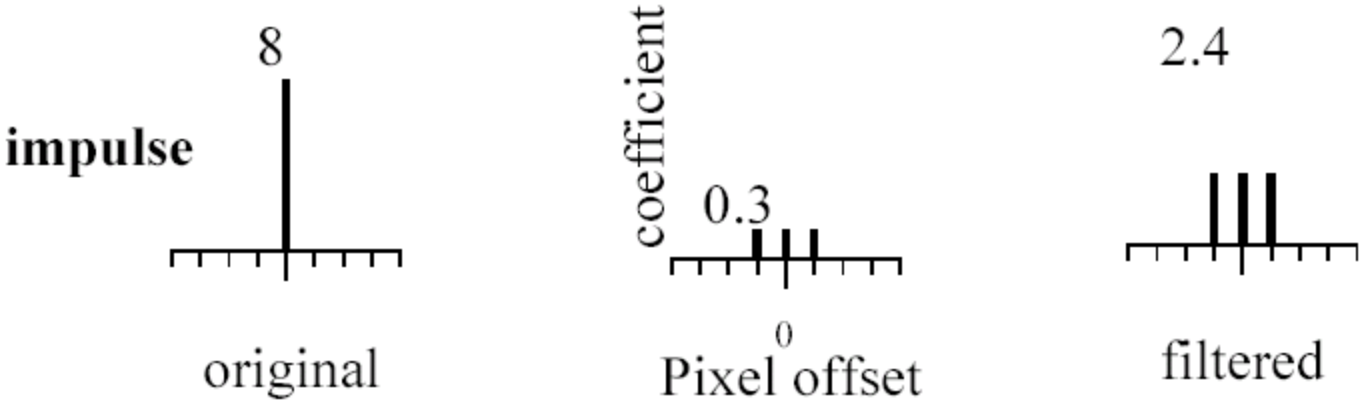
original



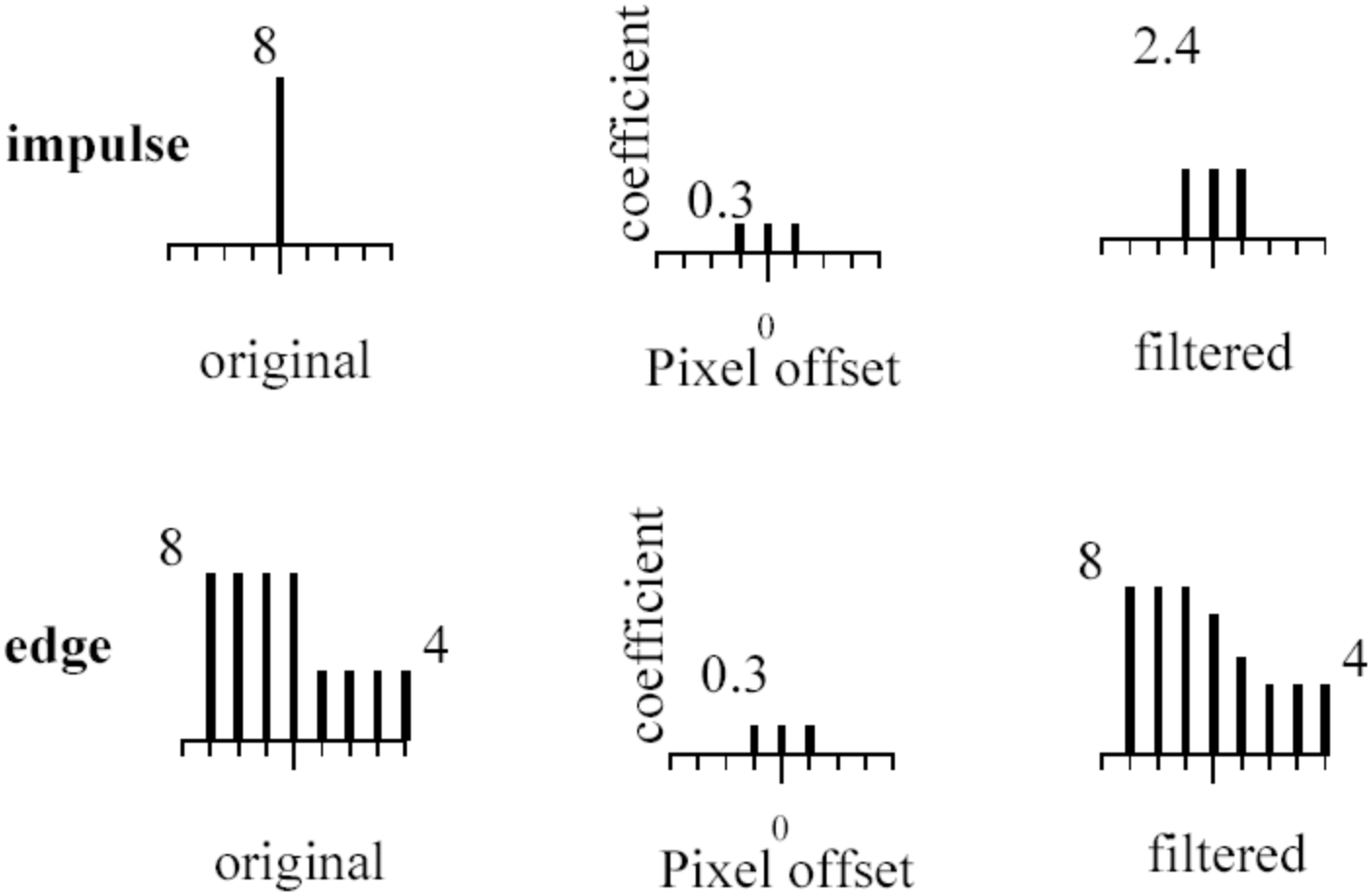


Blurred (filter applied in both dimensions).

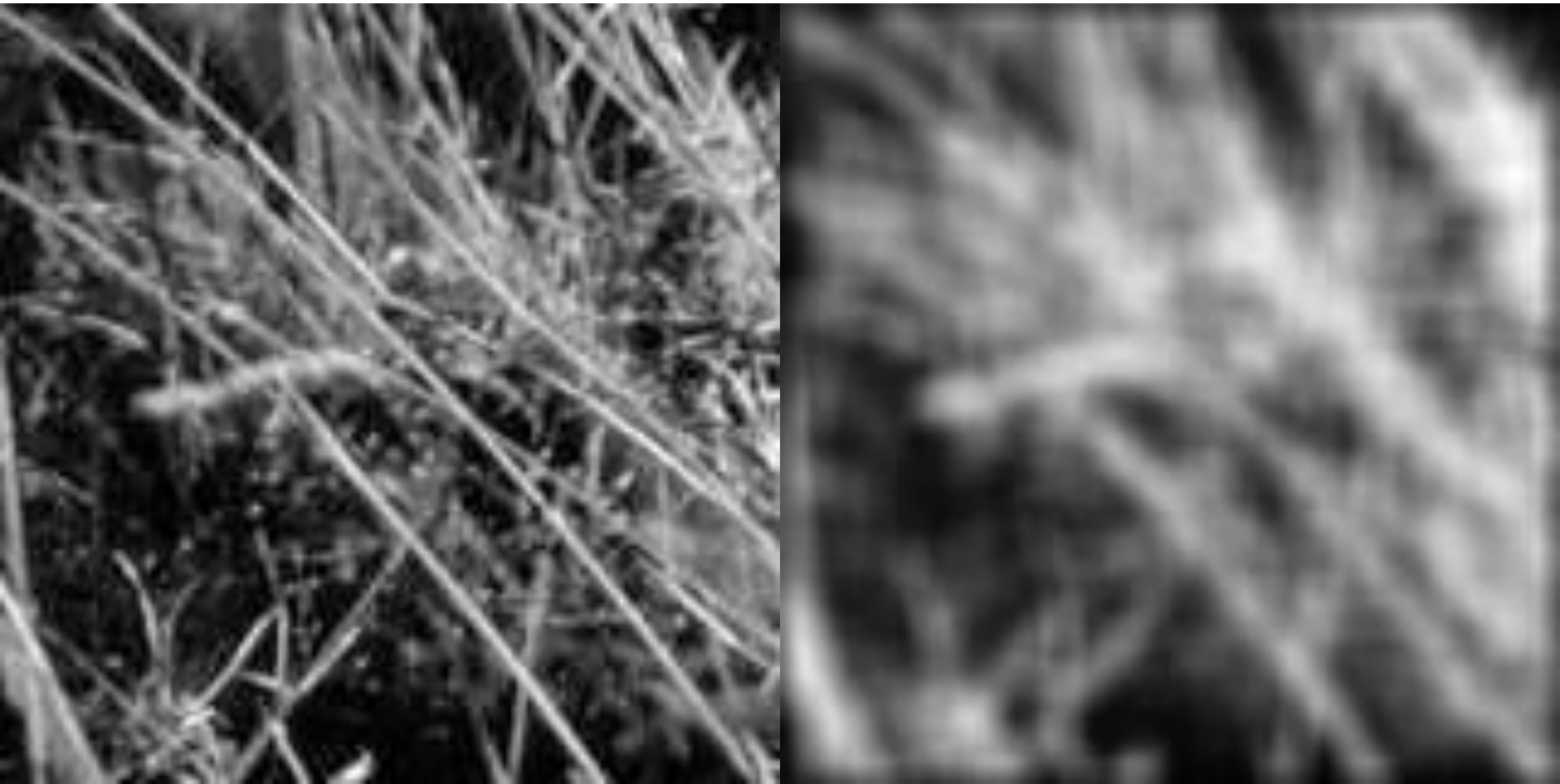
Blur examples



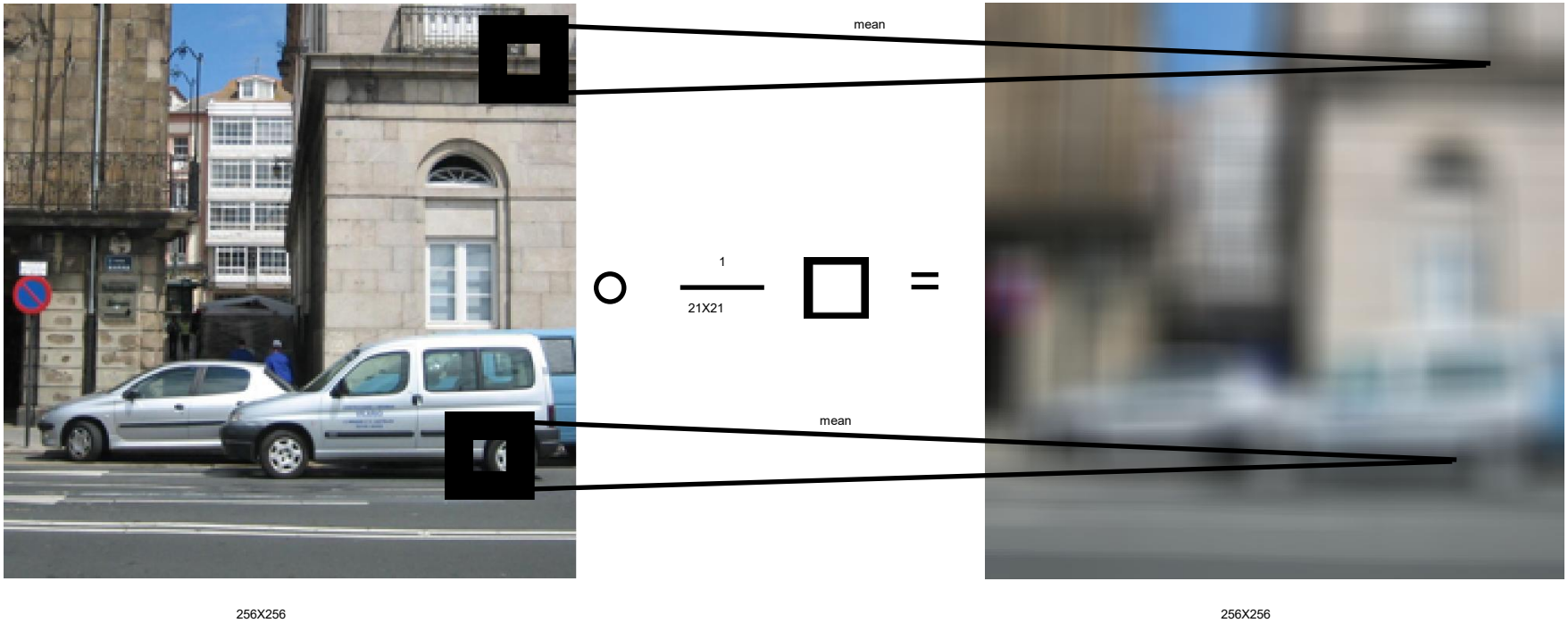
Blur examples



Example: Smoothing by Averaging



Box filter



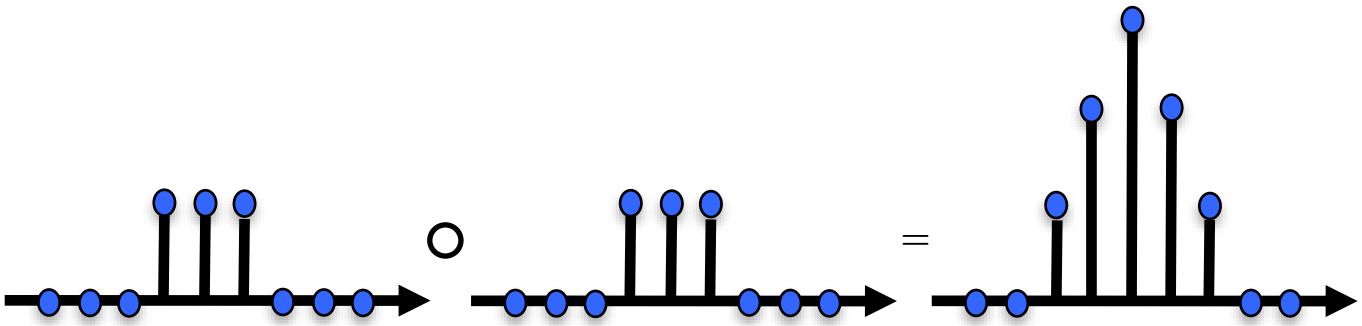
What does it do?

- Replaces each pixel with an average of its neighborhood
- Achieves smoothing effect (removes sharp, high-contrast features)

Box filter

If you convolve two boxes:

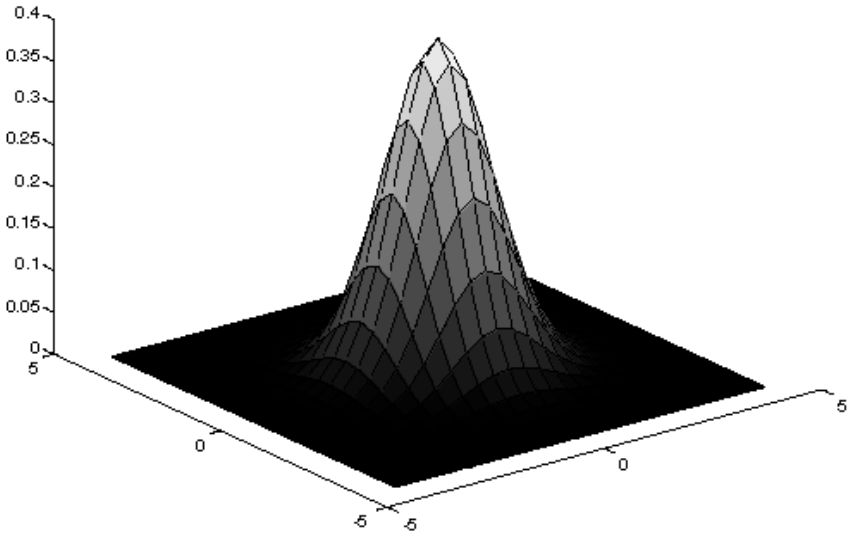
$$1\ 1\ 1 \circ 1\ 1\ 1 = 1\ 2\ 3\ 2\ 1$$



The convolution of two box filters is not another box filter.
It is a triangular filter.

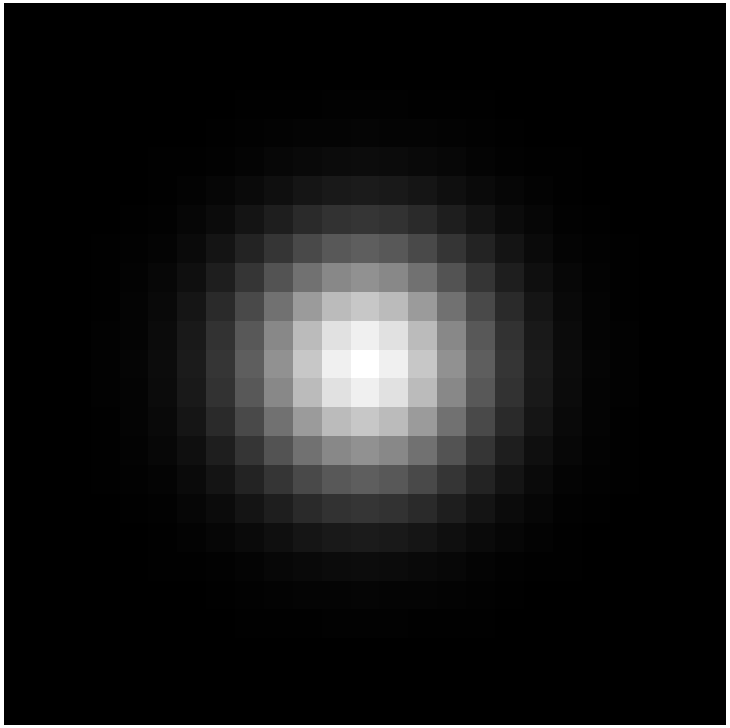
Smoothing with a Gaussian

- Smoothing with the average does not resemble a defocused lens
- One obvious difference is that a single point of light viewed in a defocused lens looks like a fuzzy blob; but the averaging process would result in a little square.



- A Gaussian gives a better model of a fuzzy blob

An Isotropic Gaussian

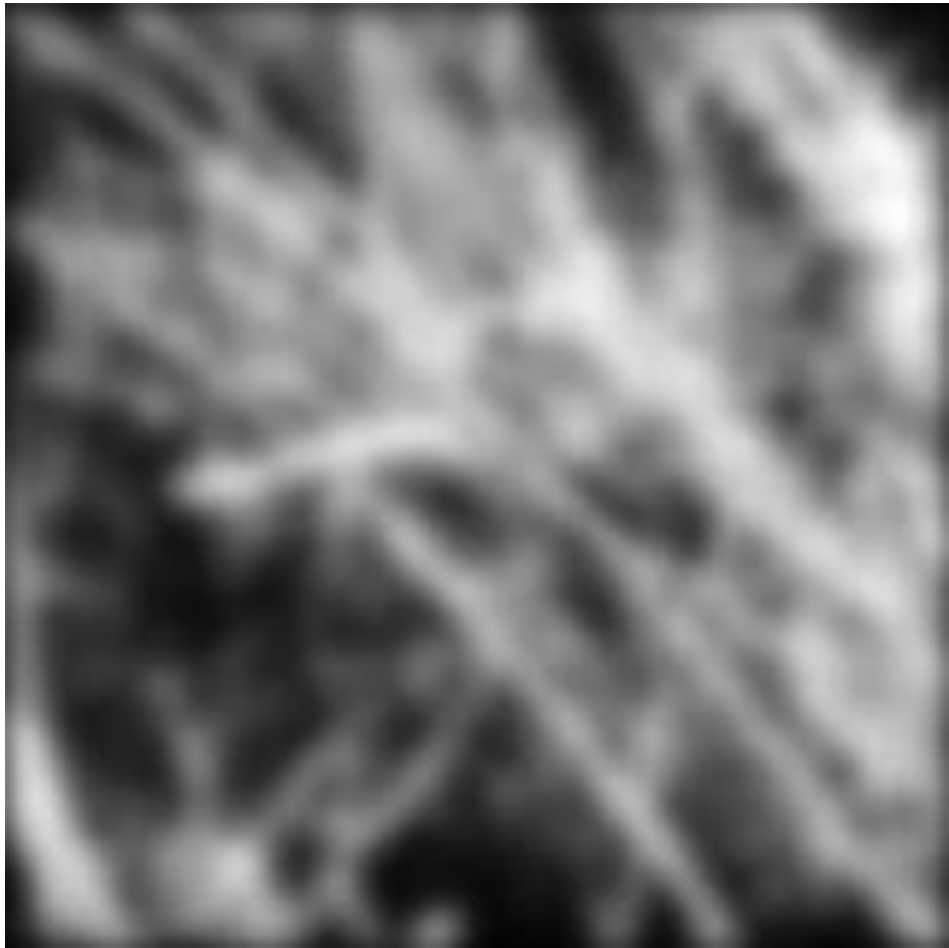


- The picture shows a smoothing kernel proportional to

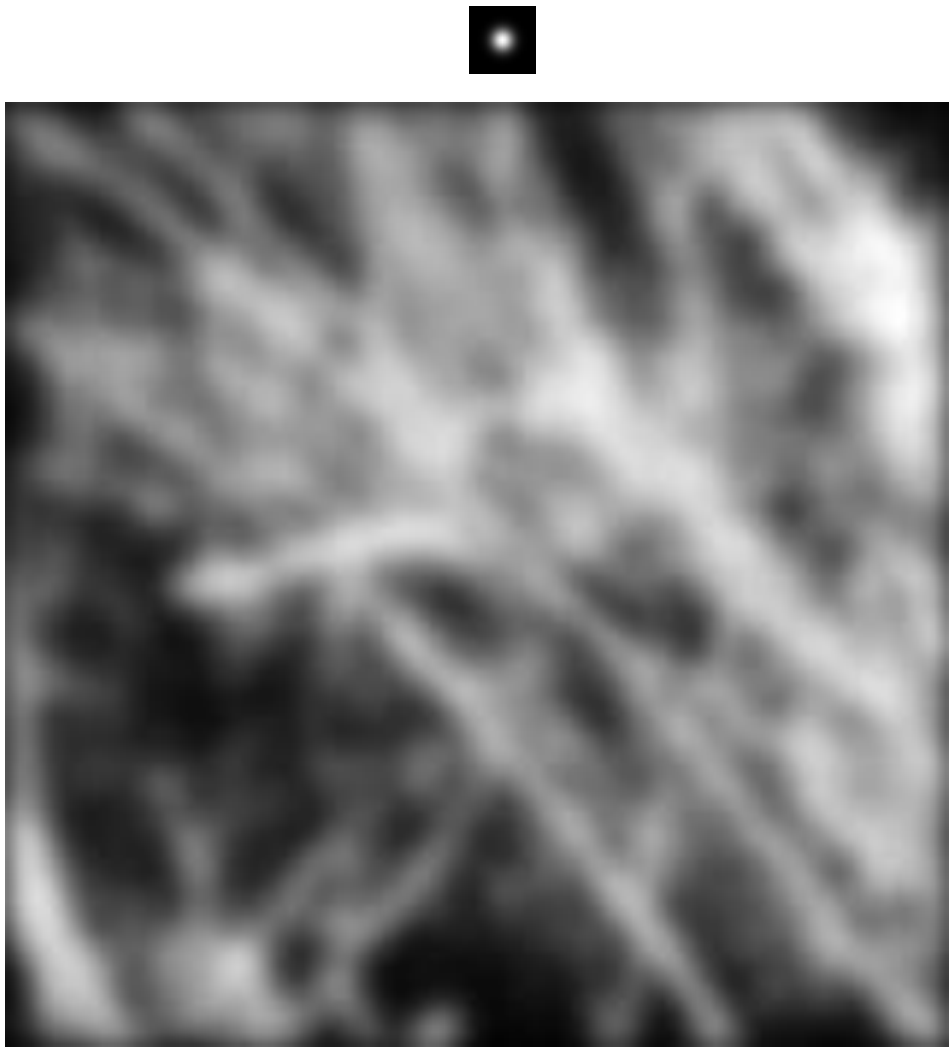
$$\exp\left(-\left(\frac{x^2+y^2}{2\sigma^2}\right)\right)$$

(which is a reasonable model of a circularly symmetric fuzzy blob)

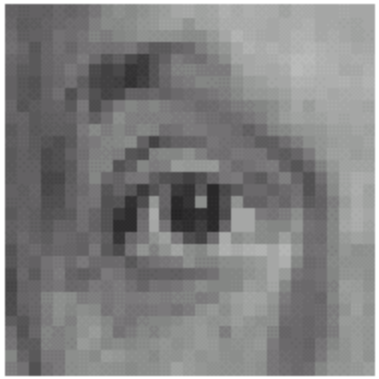
Smoothing with a Gaussian



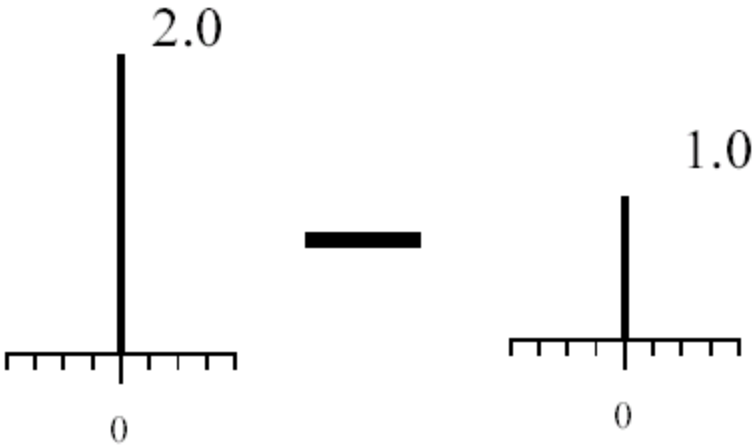
Box Filter vs. Gaussian



Linear filtering (warm-up slide)

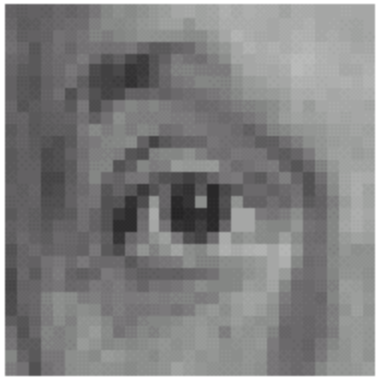


original

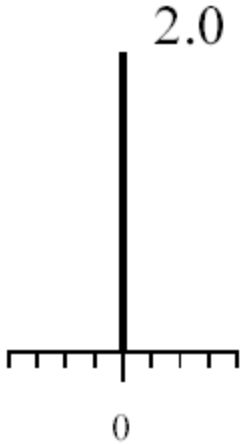


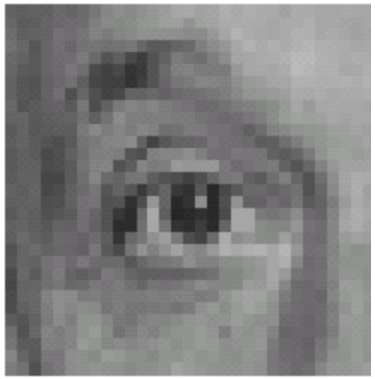
?

Linear filtering (no change)



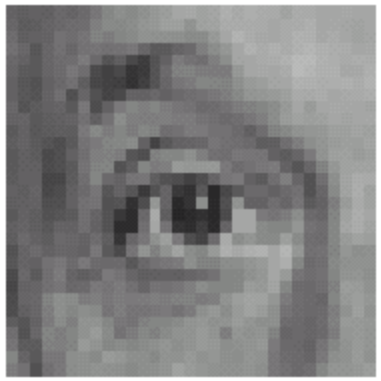
original



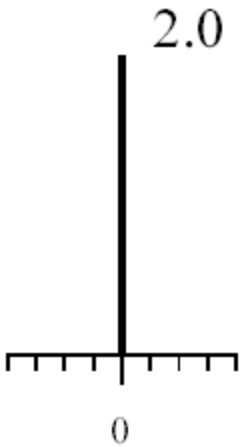


Filtered
(no change)

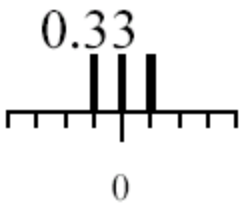
Linear filtering



original

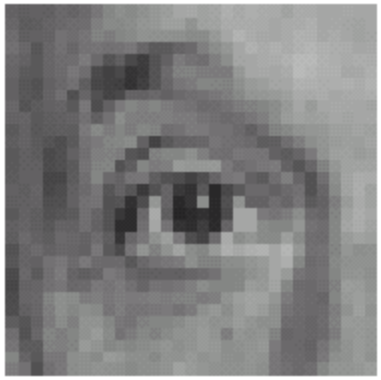


−

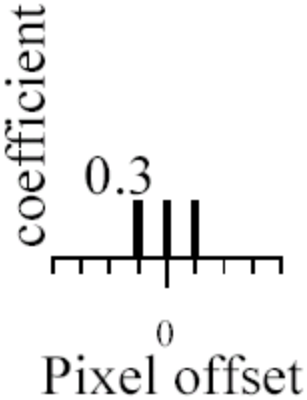


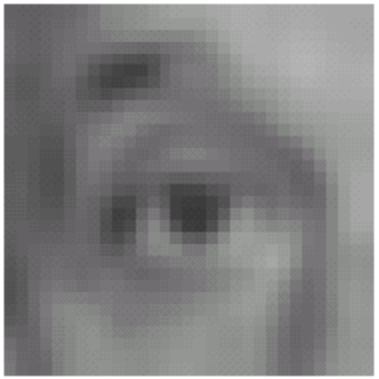
?

(remember blurring)



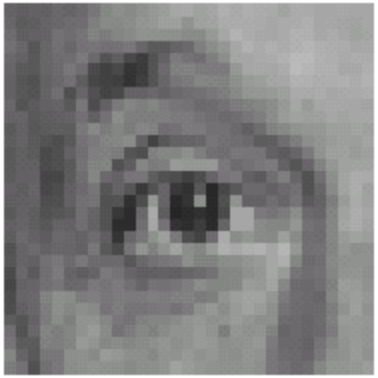
original



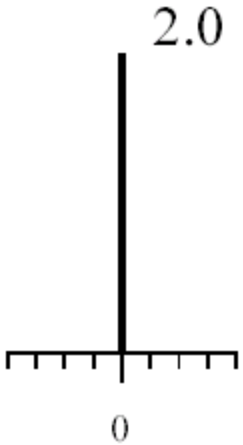


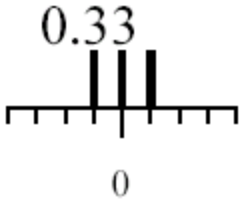
Blurred (filter applied in both dimensions).

Sharpening



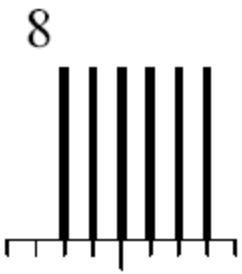
original



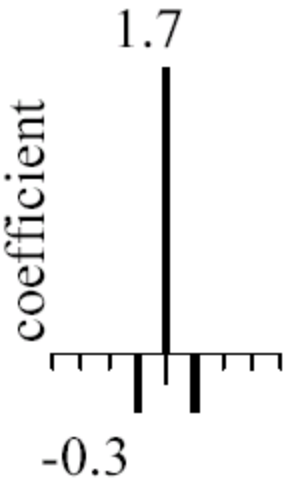


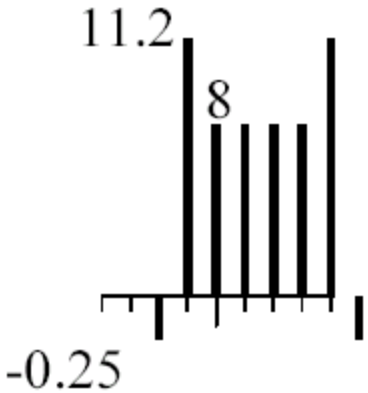
Sharpened original

Sharpening example



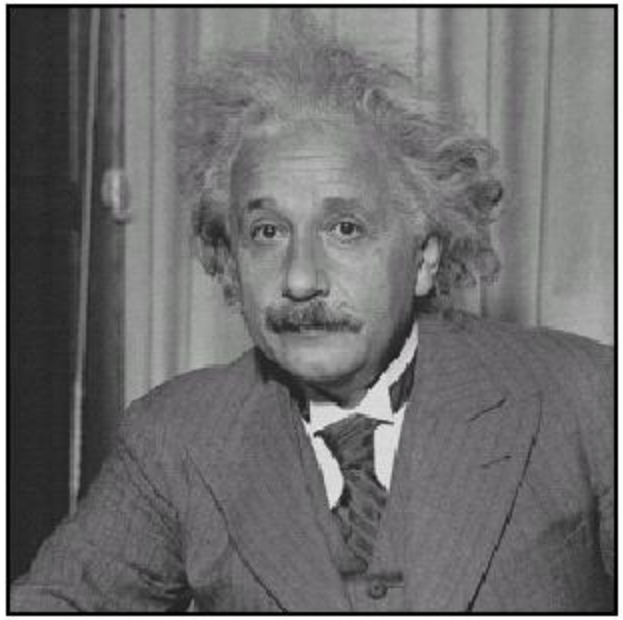
original



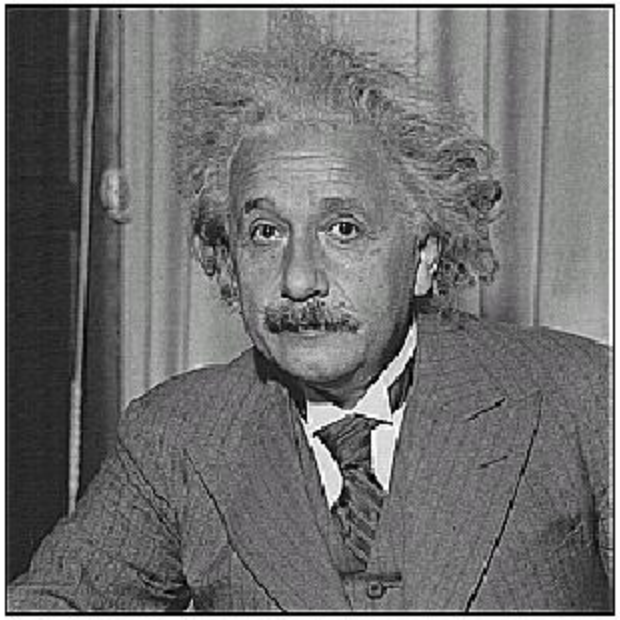


Sharpened (differences are accentuated; constant areas are left untouched).

Sharpening



before



after

unsharp masking:

$$f_{sharper}(x,y) = f(x,y) - f_{smoothed}(x,y)$$

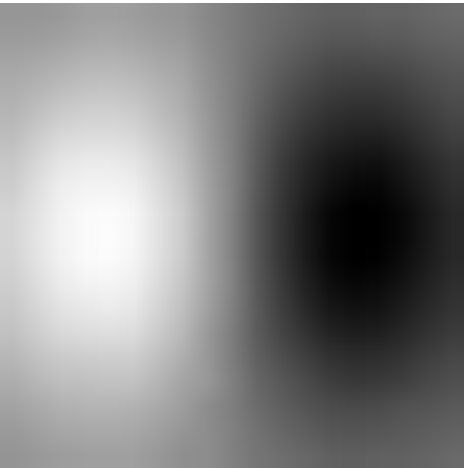
Correlation and Convolution

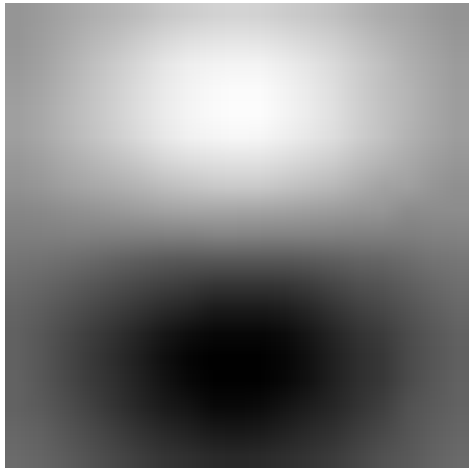
- Notation: $g = f \otimes h$ g = f * h
- Commutative?
- Associative?
- Superposition?
- Shift invariance:

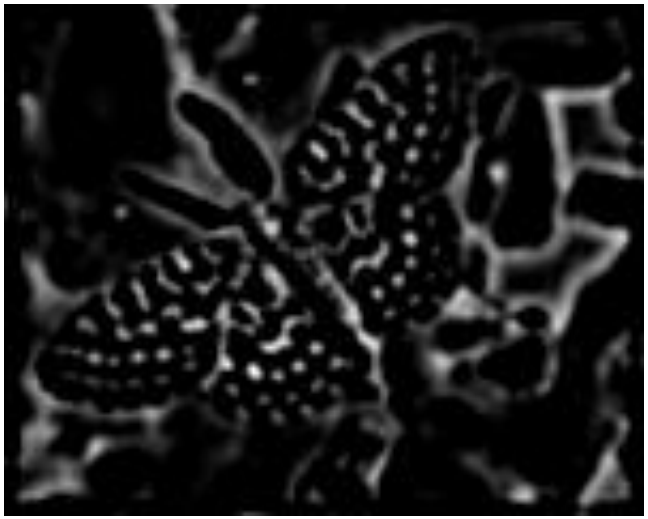
Filters are templates

- Applying a filter at some point can be seen as taking a dot-product between the image and some vector
- Filtering the image is a set of dot products

- Insight
  - filters look like the effects they are intended to find
  - filters find effects they look like



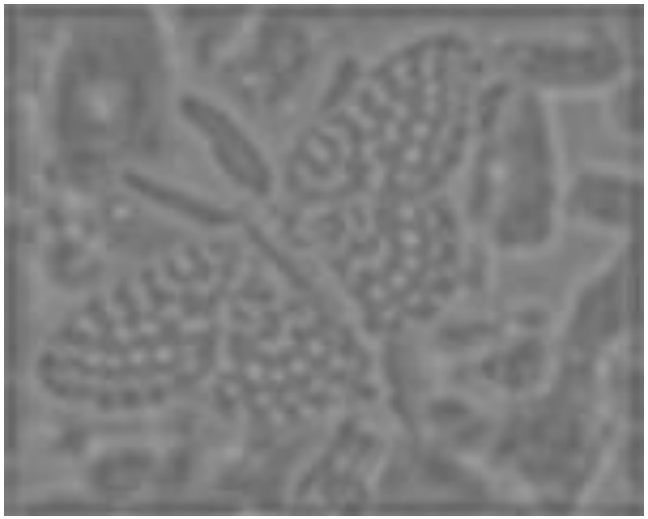




Positive responses

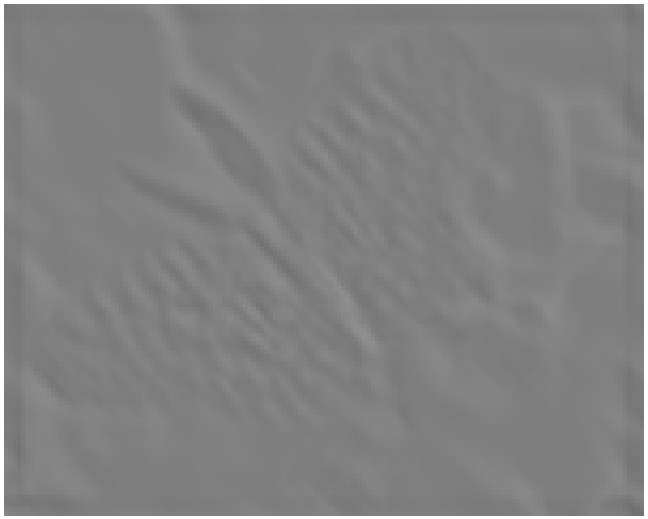
Zero mean image, -1:1 scale

Zero mean image, -max:max scale

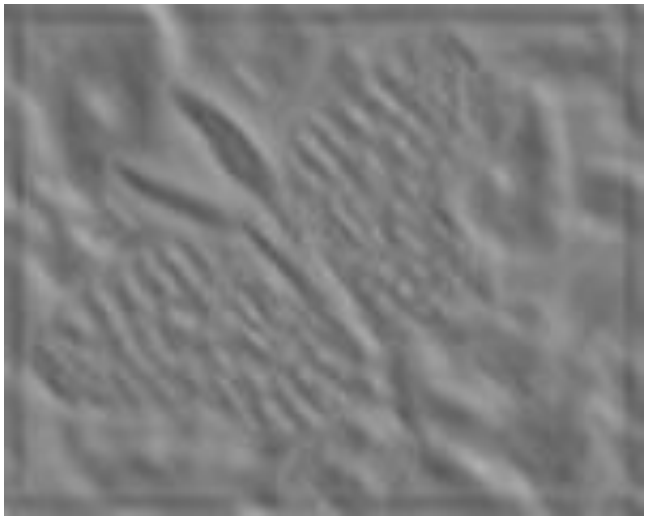


Positive responses

Zero mean image, -1:1 scale



Zero mean image, -max:max scale



Separable Filters

- Computational complexity of convolution?
- Some filters can be applied first horizontally then vertically
  - $K=vh^T$
- But consider
  - Memory locality
  - Parallelization
- Use a library and pre-defined kernels

Box filter

The box filter is separable as it can be written as the convolution of two 1D kernels

$$h_{N,M}[n,m] = h_{N,0} \circ h_{0,M}$$

$$\begin{matrix} 1 \\ 1 \\ 1 \end{matrix} \circ \begin{matrix} 1 & 1 \end{matrix} = \begin{matrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{matrix}$$

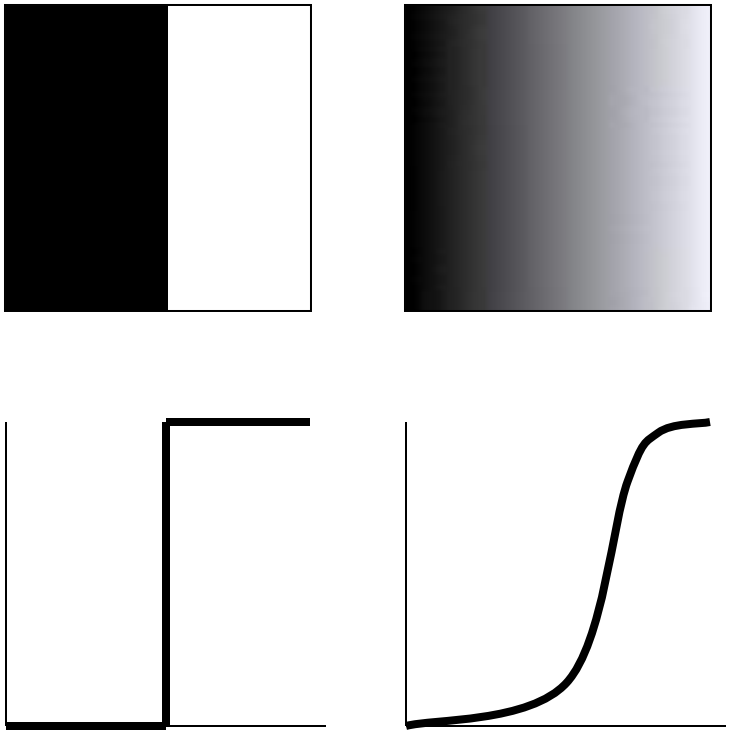
Box filter

Requires N+N sums, instead of N*N

Edge Detection with Filters

What are edges in an image?

- Edges are those places in an image that correspond to object boundaries.
- Edges are pixels where image brightness changes abruptly.
- An edge is a property attached to an individual pixel and is calculated from the image function behavior in a neighborhood of the pixel.
- It is a **vector variable** (**magnitude** of the gradient, **direction** of an edge) .



Brightness vs. Spatial Coordinates

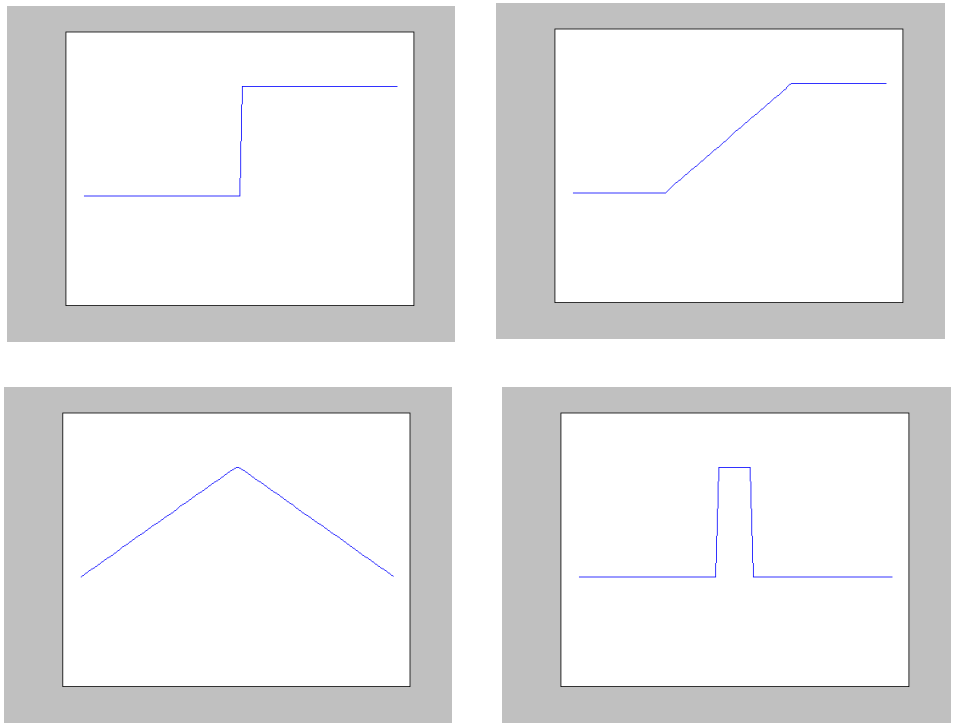
Gradients and edges

- Points of sharp change in an image are interesting:
  - change in reflectance
  - change in object
  - change in illumination
  - noise
- Sometimes called **edge points**

- General strategy
  - determine image gradient
  - now mark points where gradient magnitude is particularly large wrt neighbors (ideally, curves of such points).

What types of edge are there?

- Let us define an edge to be an approximate discontinuity in image intensity function.
- Edge Models
  - Step Edge
  - Ramp Edge
  - Roof Edge
  - Spike Edge



- Colors?
- 2D issues and Corners?

Differentiation and convolution

Discontinuities in a signal can be detected by computing the derivative of the signal.

- Recall

$$\frac{\partial f}{\partial x} = \lim_{\varepsilon \to 0}\left(\frac{f(x+\varepsilon) - f(x)}{\varepsilon}\right)$$

- This is linear and shift invariant, so must be the result of a convolution.

- We could approximate this as

$$\frac{\partial f}{\partial x} \approx \frac{f(x+1) - f(x)}{(x+1) - (x)}$$

which is a convolution with Kernel

[1 -1]

An alternative notation:

$$\frac{\partial \mathbf{I}}{\partial x} \simeq \mathbf{I}(x,y) - \mathbf{I}(x-1,y)$$

Finite Difference in 2D

$$\frac{\partial f(x,y)}{\partial x} = \lim_{\varepsilon \to 0}\left(\frac{f(x+\varepsilon,y) - f(x,y)}{\varepsilon}\right)$$

$$\frac{\partial f(x,y)}{\partial y} = \lim_{\varepsilon \to 0}\left(\frac{f(x,y+\varepsilon) - f(x,y)}{\varepsilon}\right)$$

Definition

$$\frac{\partial f(x,y)}{\partial x} \approx \frac{f(x_{n+1},y_m) - f(x_n,y_m)}{\Delta x}$$

$$\frac{\partial f(x,y)}{\partial y} \approx \frac{f(x_n,y_{m+1}) - f(x_n,y_m)}{\Delta y}$$

Discrete Approximation

$$\begin{bmatrix} 1 & -1 \end{bmatrix}$$

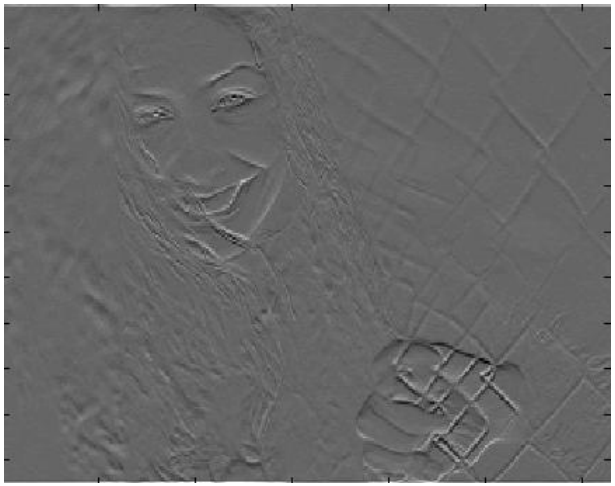
$$\begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Convolution Kernels

Finite differences

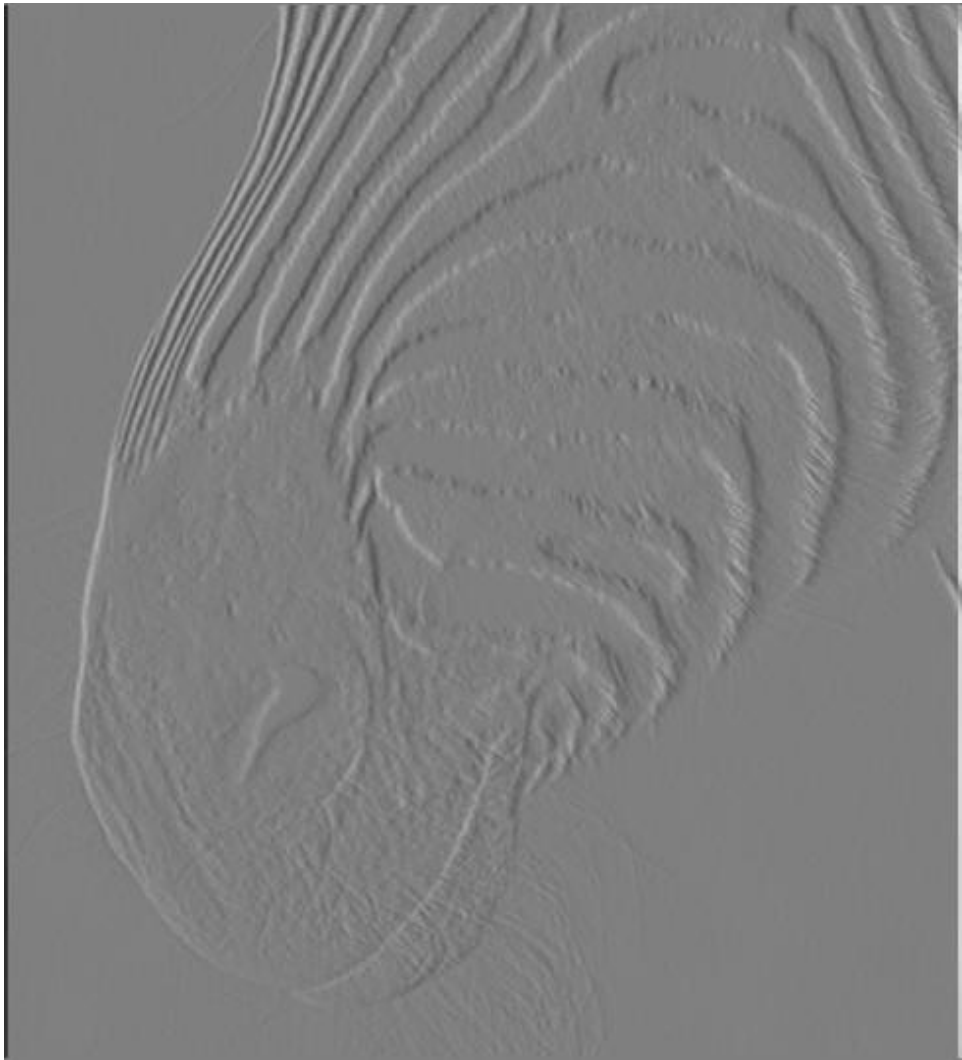
$$I_x = I * \begin{bmatrix} 1 & -1 \end{bmatrix}$$

$$I$$



$$I_y = I * \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Finite differences



Classical Operators

Robert's cross-gradient operators

$$\begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \qquad \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

Classical Operators

Prewitt's Operator

$$\longrightarrow \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & -1 \end{bmatrix}$$

Smooth

Differentiate

$$\longrightarrow \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Classical Operators

Sobel's Operator

$$\longrightarrow \begin{bmatrix} 1 & 1 \\ 2 & 2 \\ 1 & 1 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & -1 \end{bmatrix}$$

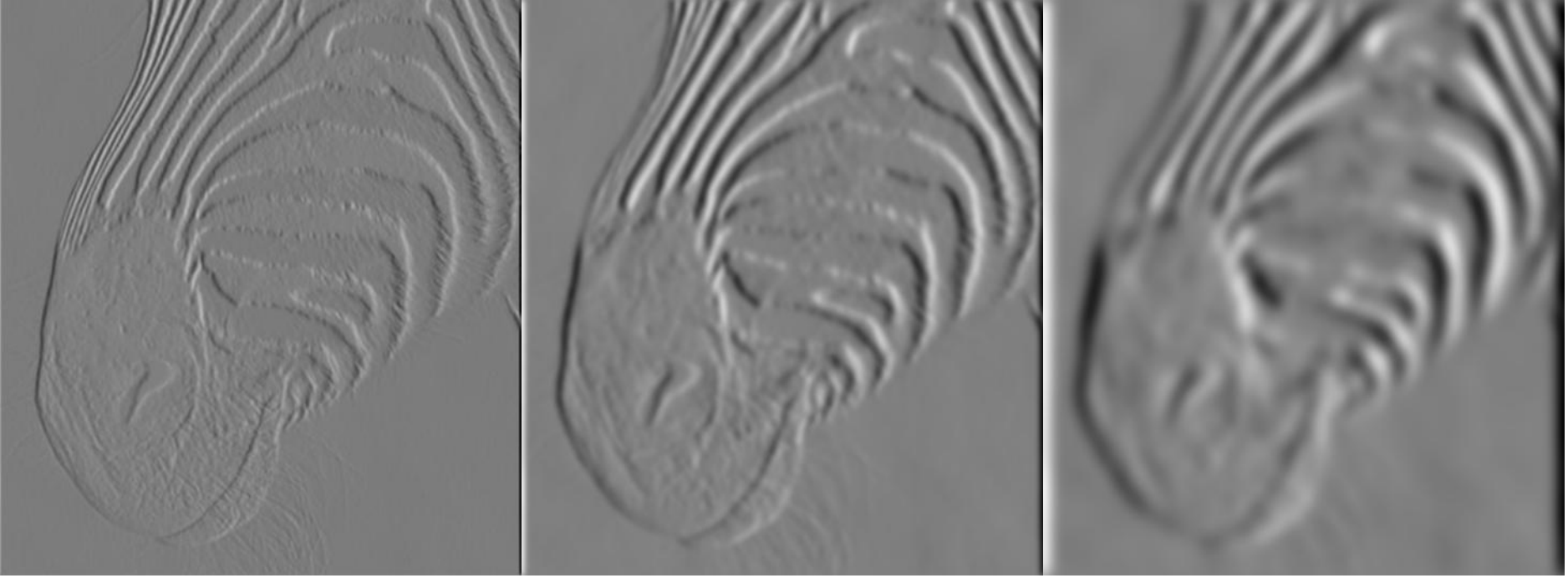
Smooth

Differentiate

$$\longrightarrow \begin{bmatrix} 1 & 2 & 1 \\ 1 & 2 & 1 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$



1 pixel

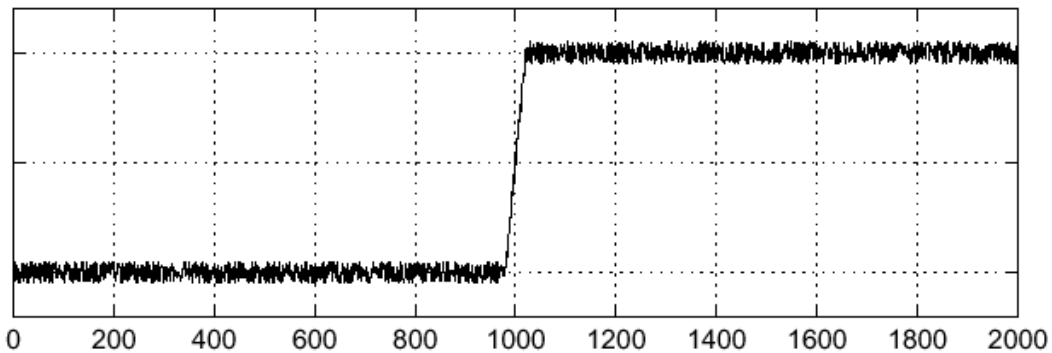
3 pixels

7 pixels

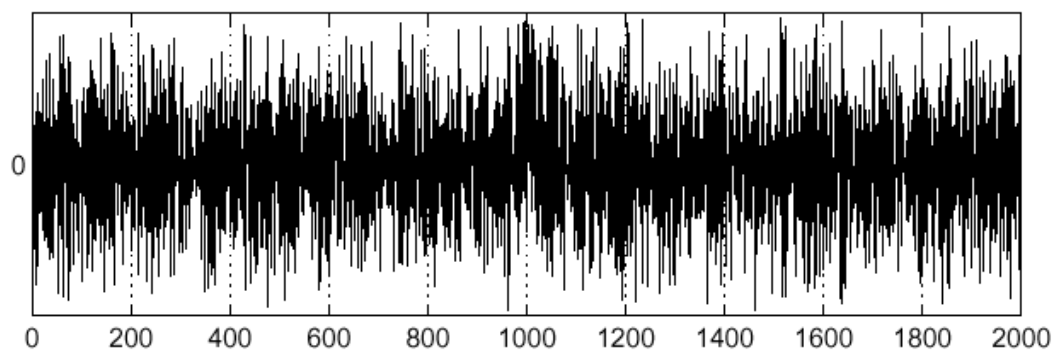
The scale of the smoothing filter affects derivative estimates, and also the semantics of the edges recovered.

Image Noise and Gradients

- Consider a single row or column of the image
- Plotting intensity as a function of position gives a signal



$$\frac{d}{dx}f(x)$$



Where is the edge?

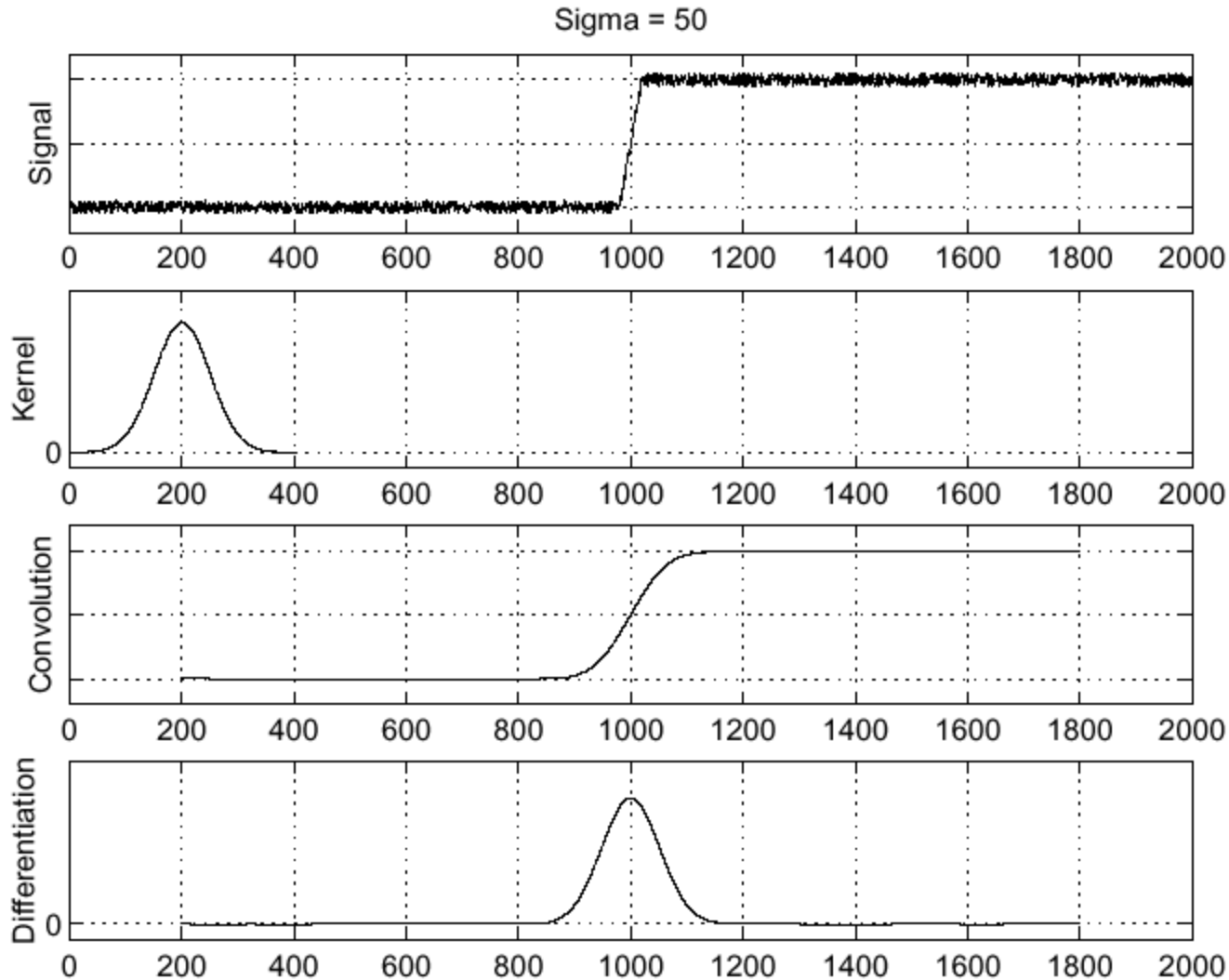
Solution: Smooth, then Differentiate

f

h

$h*f$

$\frac{\partial}{\partial x}(h \star f)$



Where is the edge?

Look for peaks in $\frac{\partial}{\partial x}(h \star f)$

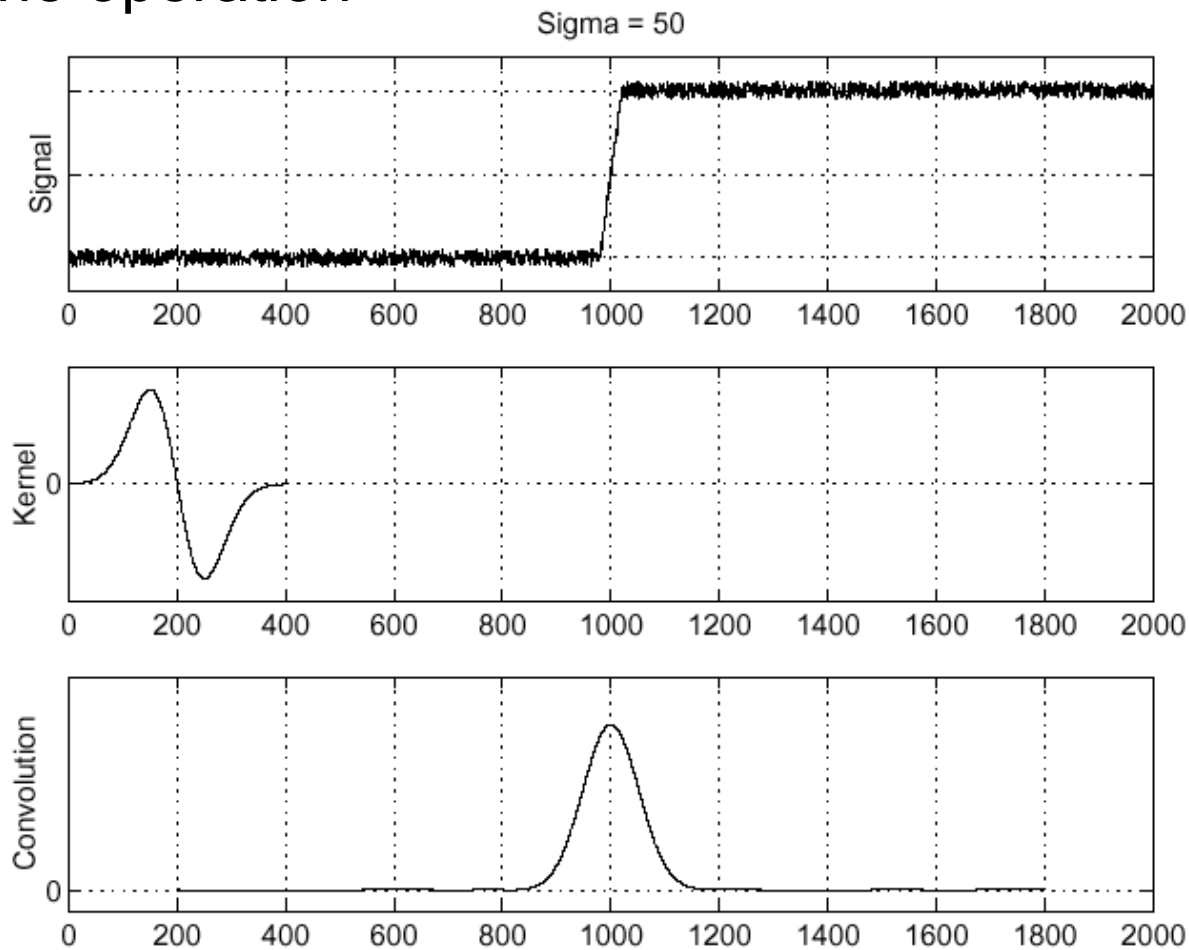
Convolution is Associative

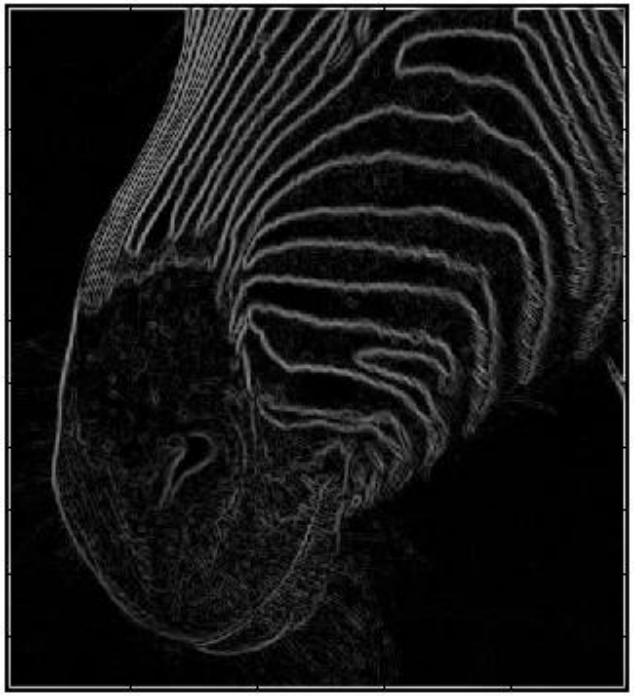
$$\frac{\partial}{\partial x}(h \star f) = (\frac{\partial}{\partial x}h) \star f$$

This saves one operation

$$\frac{\partial}{\partial x}h$$

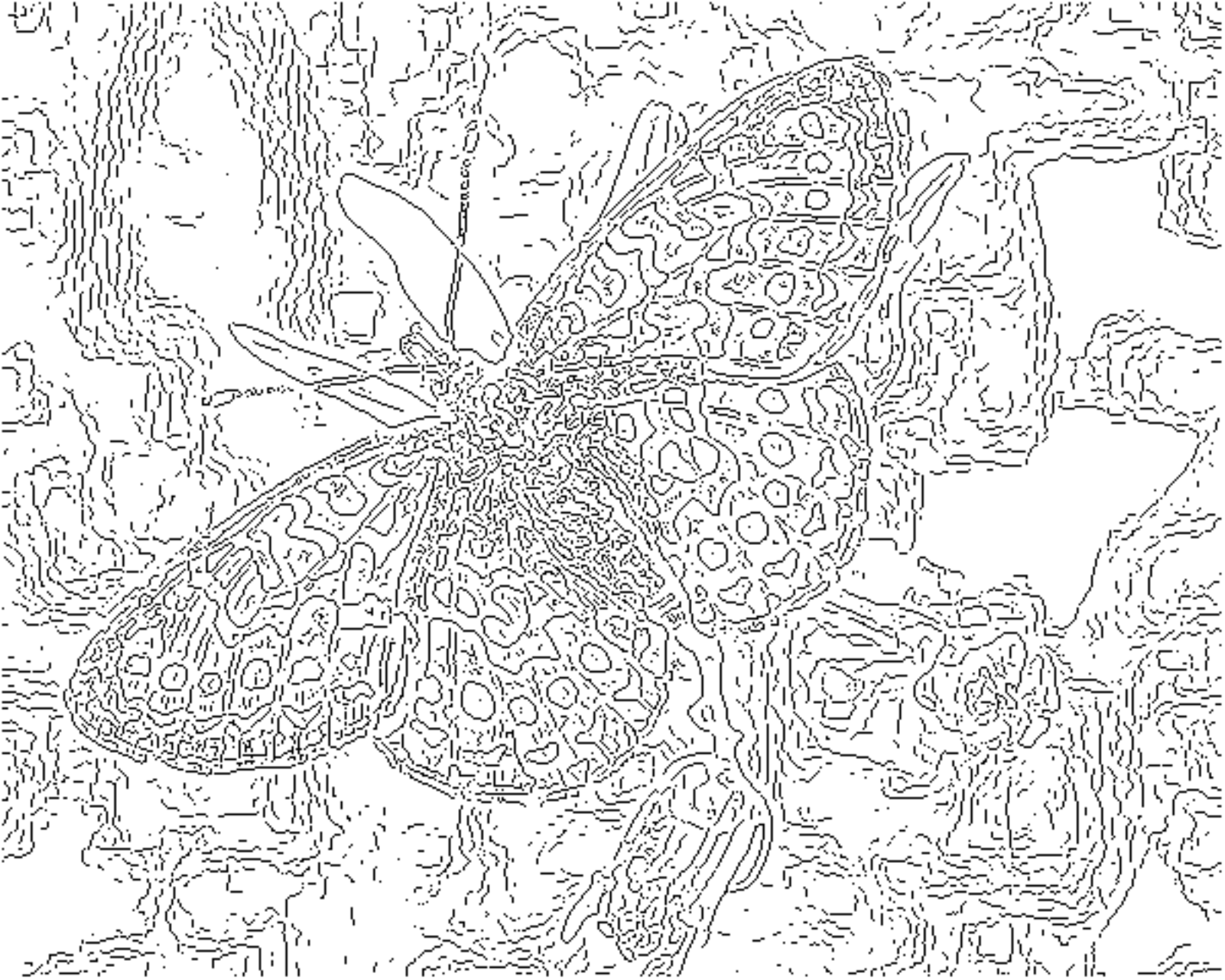
$$(\frac{\partial}{\partial x}h) \star f$$





There are three major issues:

1) The gradient magnitude at different scales is different; which should we choose?
2) The gradient magnitude is large along thick trail; how do we identify the significant points?
3) How do we link the relevant points up into curves?



fine scale
high
threshold

coarse scale, high threshold

coarse
scale
low
threshold

Edge Detection, continued

- How else can we find extremal values of the first derivative?

2nd Derivative – Laplacian

- magnitude of 2nd derivative:

$$\nabla \cdot \nabla \, f = \nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2}$$

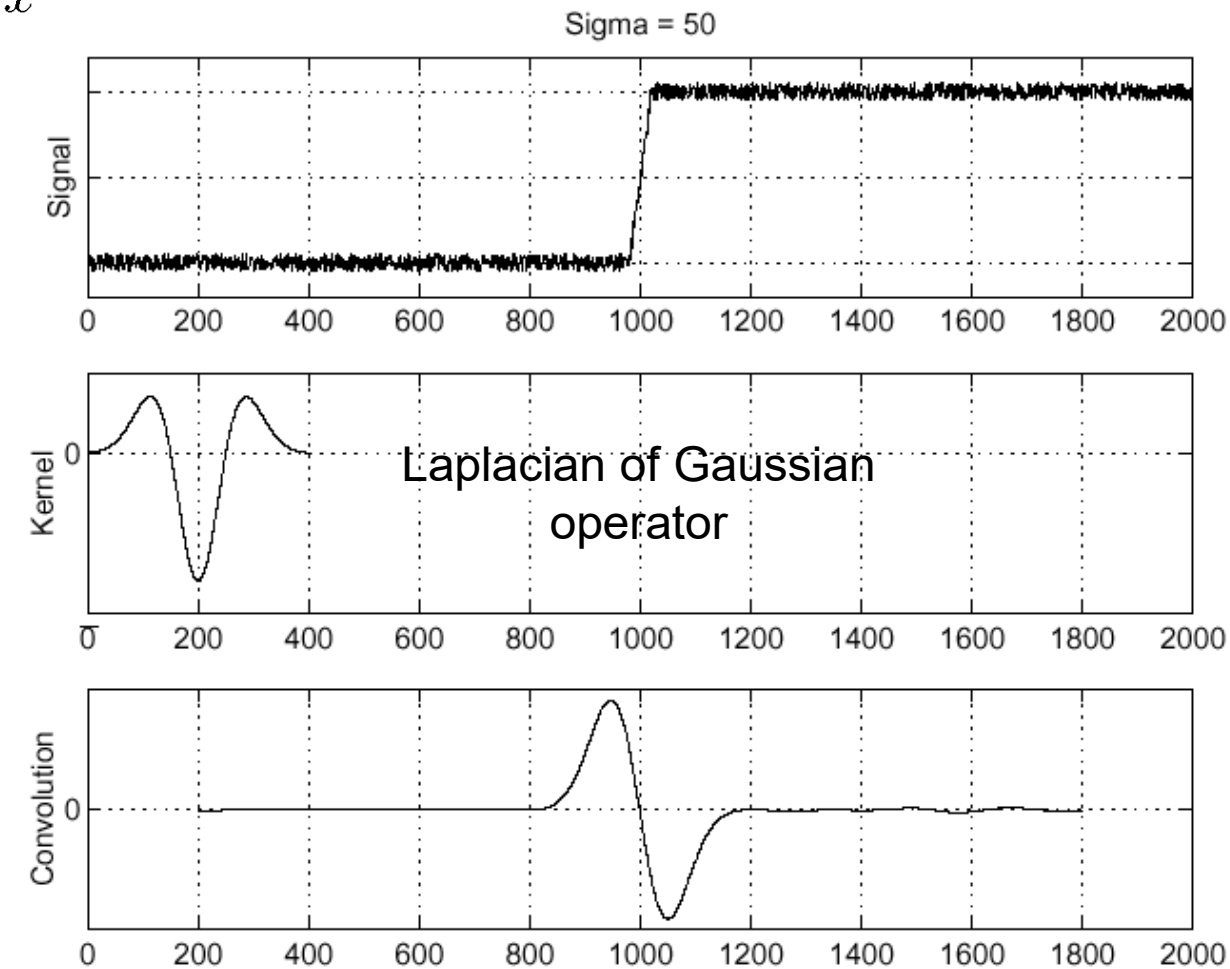
(we don't care about direction at this time)

Laplacian of Gaussian

$$\frac{\partial^2}{\partial x^2}(h \star f)$$

$$\frac{\partial^2}{\partial x^2}h$$

$$(\frac{\partial^2}{\partial x^2}h) \star f$$



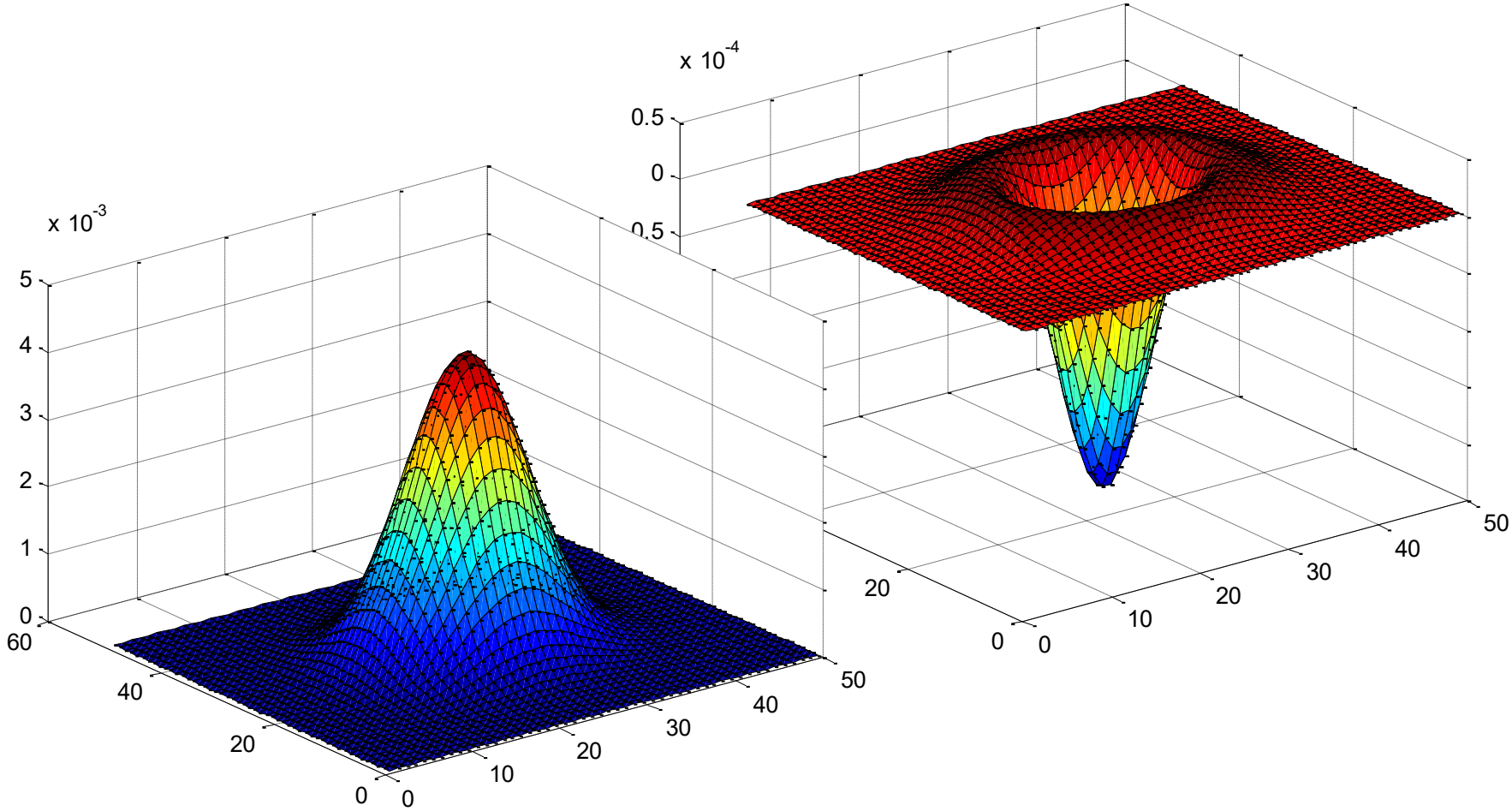
Where is the edge?

Zero-crossings of bottom graph

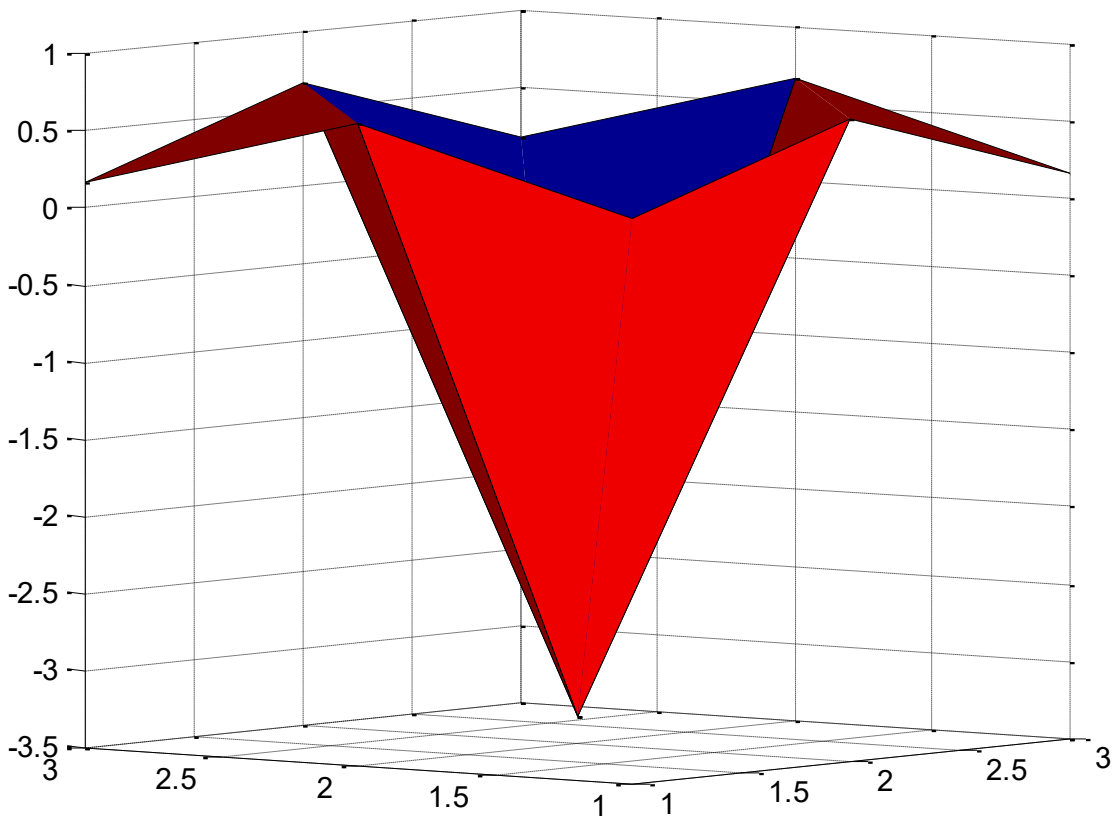
The Laplacian of Gaussian

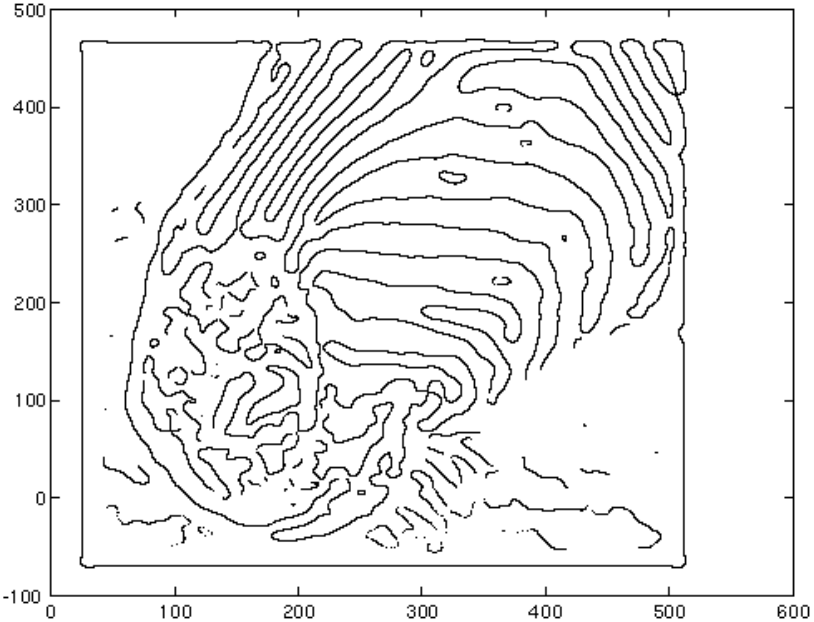
- Another way to detect an extremal first derivative is to look for a zero second derivative
- Appropriate 2D analogy is rotation invariant
  - the Laplacian
- Bad idea to apply a Laplacian without smoothing
  - **first** smooth with Gaussian, **then** apply Laplacian
  - same as filtering with a **Laplacian of Gaussian** filter
- Now mark the zero points where there is a sufficiently large derivative, and enough contrast

Laplacian of Gaussian

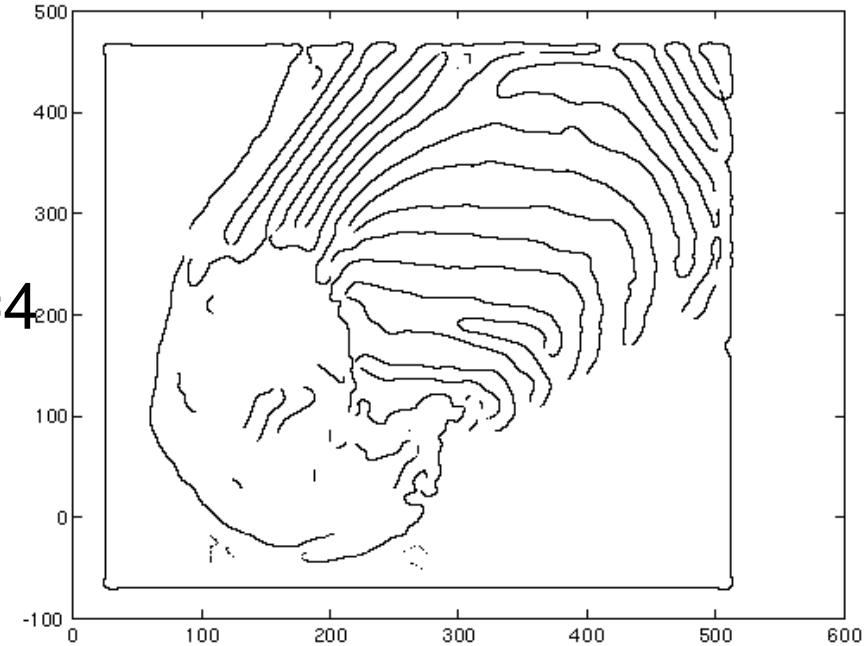


3x3 Laplacian





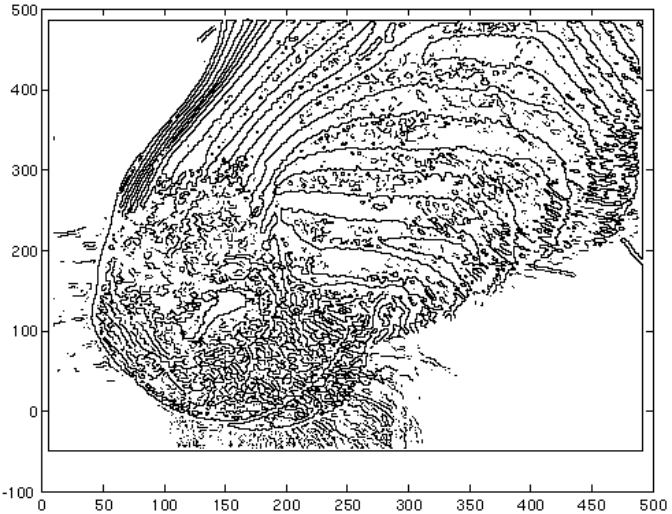
sigma=4



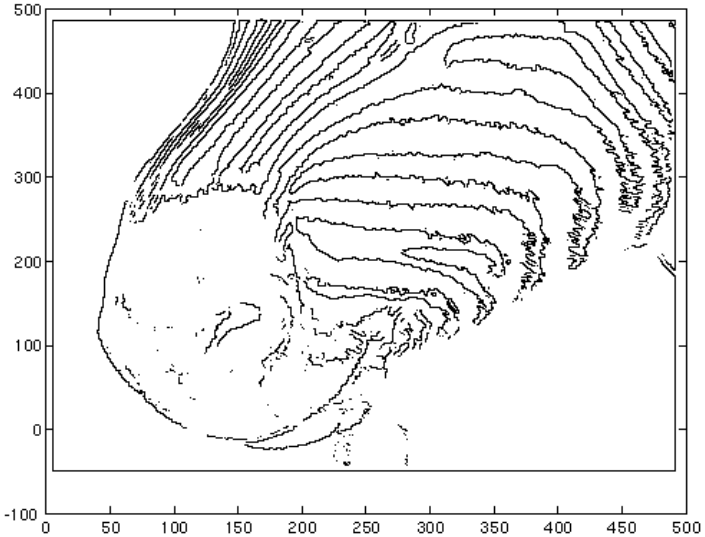
contrast=1

LoG zero crossings

contrast=4



sigma=2



Normalized (cross-)correlation

Think of filters as a dot product

- now measure the angle
- i.e *normalized* correlation output is filter output, divided by root sum of squares of values over which filter lies

Tricks:

- ensure that filter has a zero response to a constant region (helps reduce response to irrelevant background)
- subtract image average when computing the normalizing constant (i.e. subtract the image mean in the neighborhood)
- absolute value deals with contrast reversal

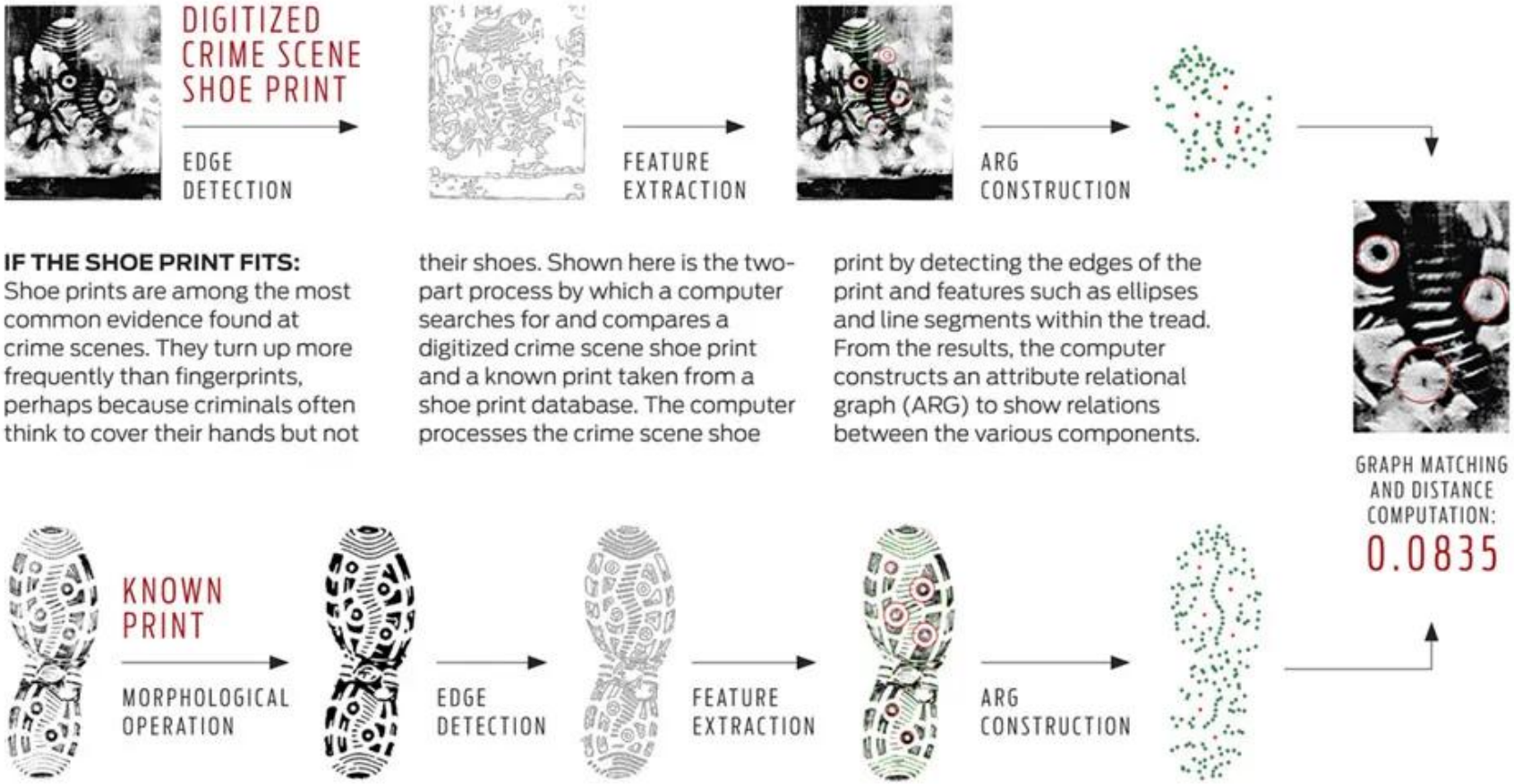
$$\gamma(u,v) = \frac{\sum_{x,y}[f(x,y) - \bar{f}_{u,v}][t(x-u,y-v) - \bar{t}]}{\left\{\sum_{x,y}[f(x,y) - \bar{f}_{u,v}]^2 \sum_{x,y}[t(x-u,y-v) - \bar{t}]^2\right\}^{0.5}}$$

Canny Edge Detector

- Probably the best yet simple edge detector
- Lots of online material for experimentation
- Incl. OpenCV Python Canny tutorial

Morphology

- Yet another image processing method
- Very limited coverage here
- Try it out in the lab if you'd like:
  - cv2.erode, cv2.dilate
  - cv2.MorphTypes



IF THE SHOE PRINT FITS: Shoe prints are among the most common evidence found at crime scenes. They turn up more frequently than fingerprints, perhaps because criminals often think to cover their hands but not their shoes. Shown here is the two-part process by which a computer searches for and compares a digitized crime scene shoe print and a known print taken from a shoe print database. The computer processes the crime scene shoe print by detecting the edges of the print and features such as ellipses and line segments within the tread. From the results, the computer constructs an attribute relational graph (ARG) to show relations between the various components.

The known shoe print is enhanced, and then, as with the crime scene print, its edges and features are detected and an ARG constructed. The two ARGs are then compared, resulting in a score ranging from 0 to 1, with 1 indicating a complete match and 0 indicating no similarity. The comparison can be done with each known print in the database, eventually resulting in a short list of prints that closely match the crime scene print.

CENTER OF EXCELLENCE FOR DOCUMENT ANALYSIS AND RECOGNITION/STATE UNIVERSITY OF NEW YORK AT BUFFALO

https://spectrum.ieee.org/beyond-csi-the-rise-of-computational-forensics

Bilateral Filtering

- Different filter shape based on image
- Optional topic: see separate slide deck

Input image | Median 3x3 | Bilateral filter 7x7 window

Image Pyramids

- Big bars and little bars are both interesting
- Inefficient to detect big bars with big filters
  - And there is superfluous detail in the filter kernel
- Alternative:
  - Apply filters of fixed size to images of different sizes
  - Typically, a collection of images whose edge length changes by a factor of 2 (or root 2)

Applications of Image Pyramids

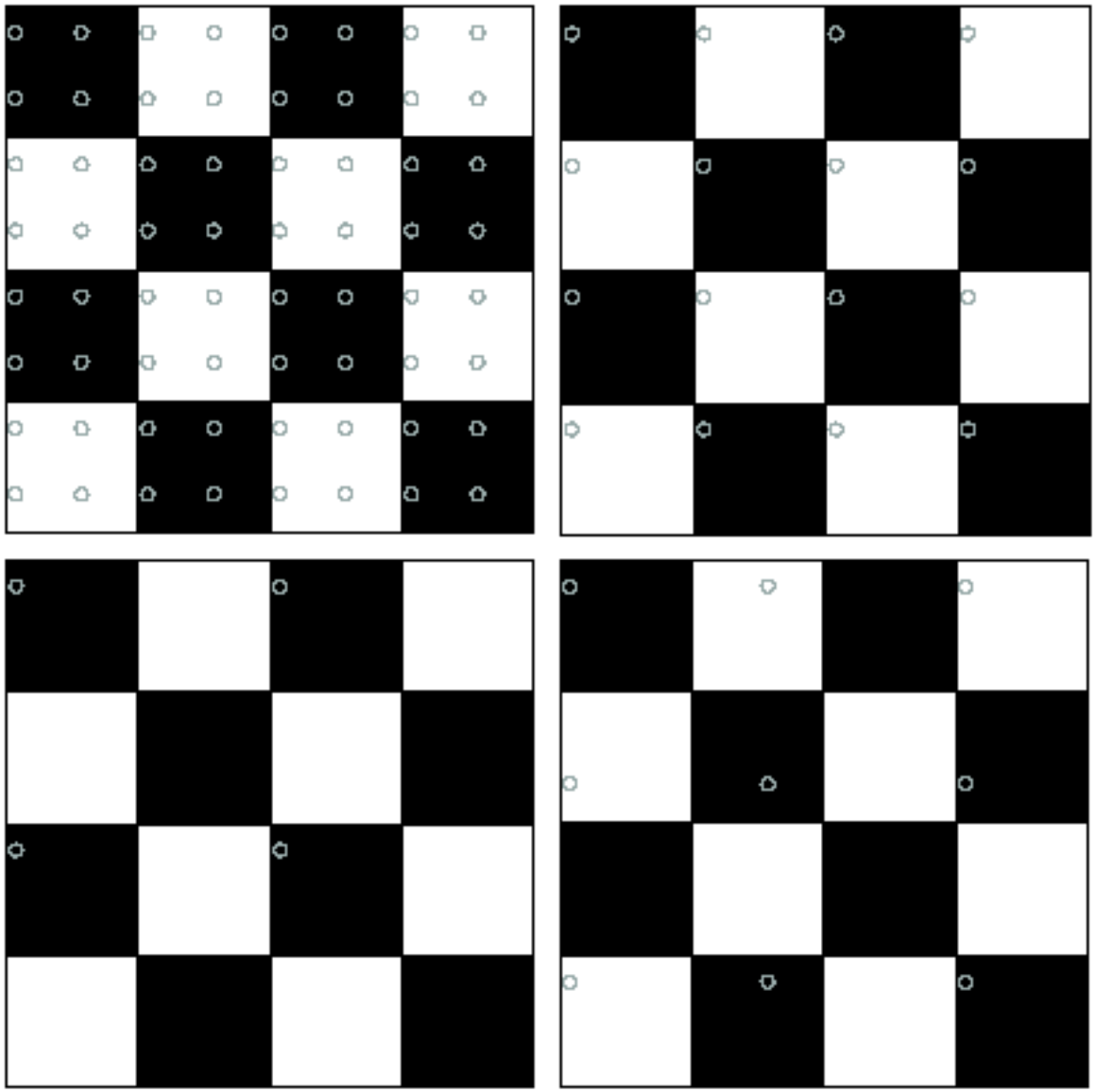
- Search for correspondence
  - look at coarse scales, then refine with finer scales
- Edge tracking
  - a “good” edge at a fine scale has parents at a coarser scale
- Control of detail and computational cost in matching
  - e.g. finding stripes
  - terribly important in texture representation

512 256 128 64 32 16 8

A bar in the big images is a hair on the zebra's nose; in smaller images, a stripe; in the smallest, the animal's nose

Aliasing

- Can’t shrink an image by taking every other pixel
- If we do, characteristic errors appear
  - Typically, small phenomena look bigger; fast phenomena can look slower
  - Common phenomenon. Examples:
    - Wagon wheels rolling the wrong way in movies
    - Checkerboards misrepresented in ray tracing
    - Striped shirts look funny on colour television

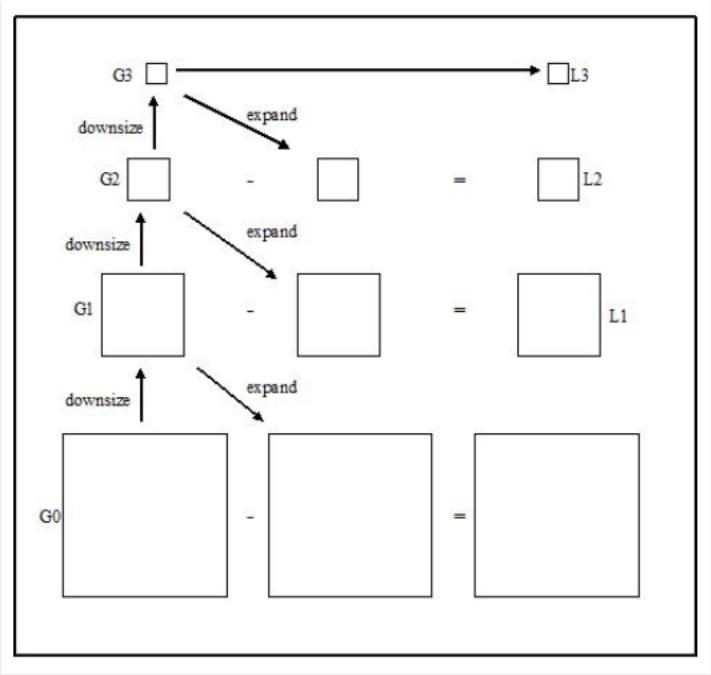


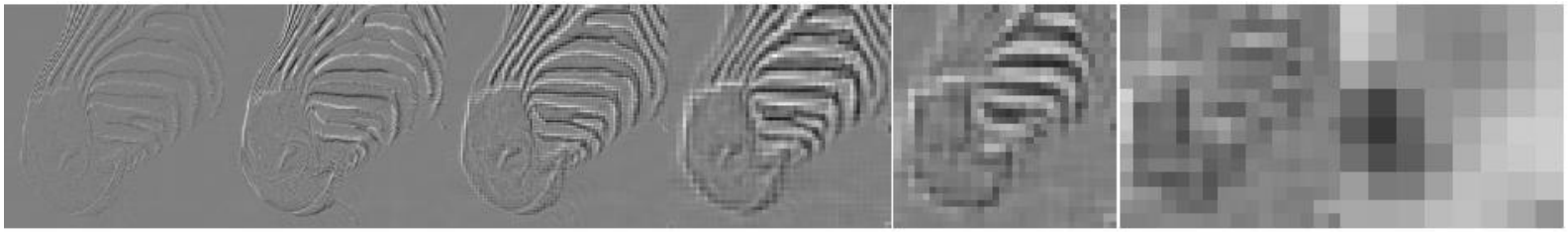
Resample the checkerboard by taking one sample at each circle. In the case of the top left board, new representation is reasonable.
Top right also yields a reasonable representation.
Bottom left is all black (dubious) and bottom right has checks that are too big.

The Laplacian Pyramid

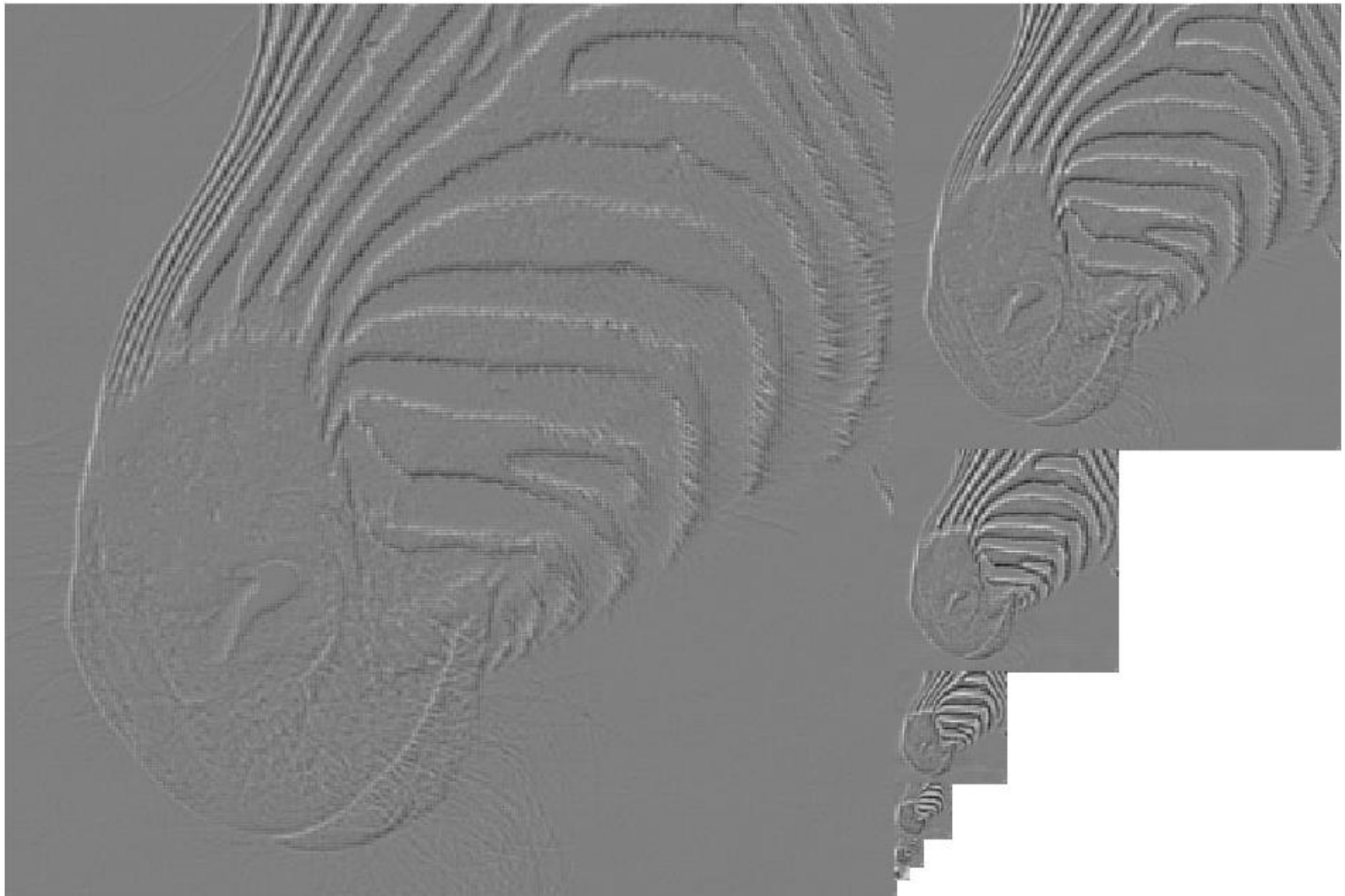
(Different concept than the Laplacian of Gaussian)

- Synthesis
  - preserve difference between upsampled Gaussian pyramid level and Gaussian pyramid level
  - band pass filter - each level represents spatial frequencies (largely) unrepresented at other levels
- Analysis
  - reconstruct Gaussian pyramid, take top layer





512 256 128 64 32 16 8

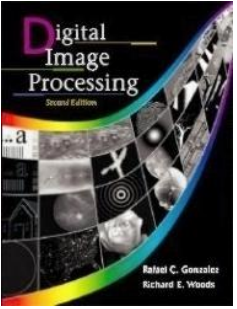


There’s also: Spectral Filtering

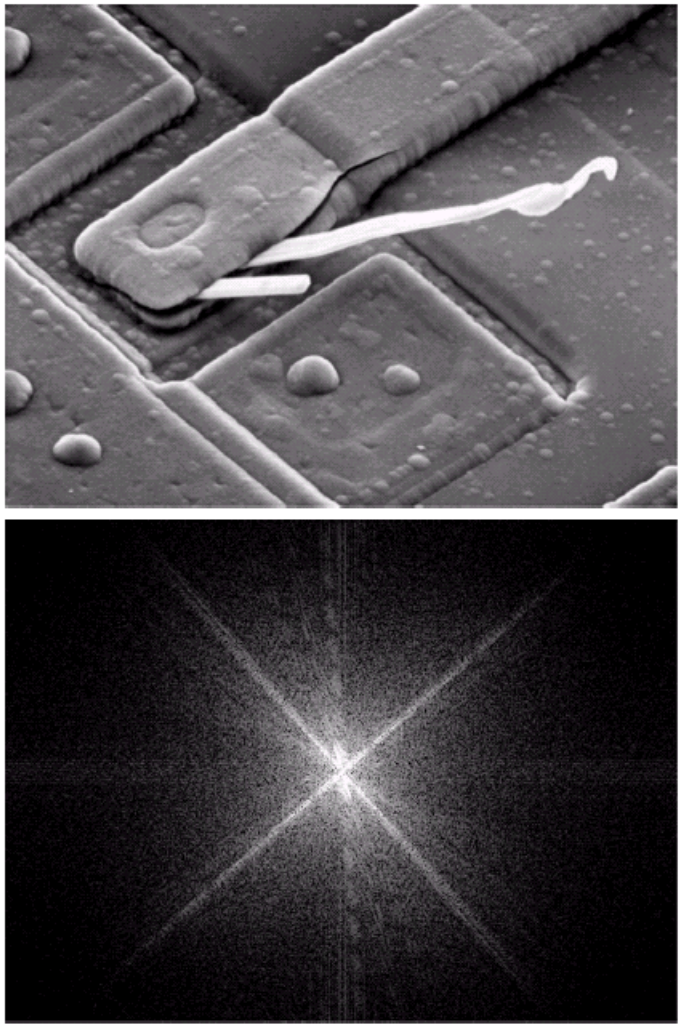
Optional topic

20 min video introduction:

https://www.youtube.com/watch?v=spUNpyF58BY

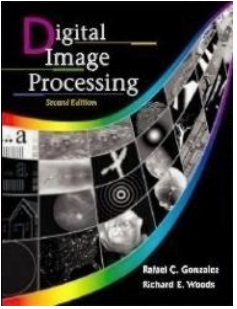


Chapter 4
Image Enhancement in the Frequency Domain

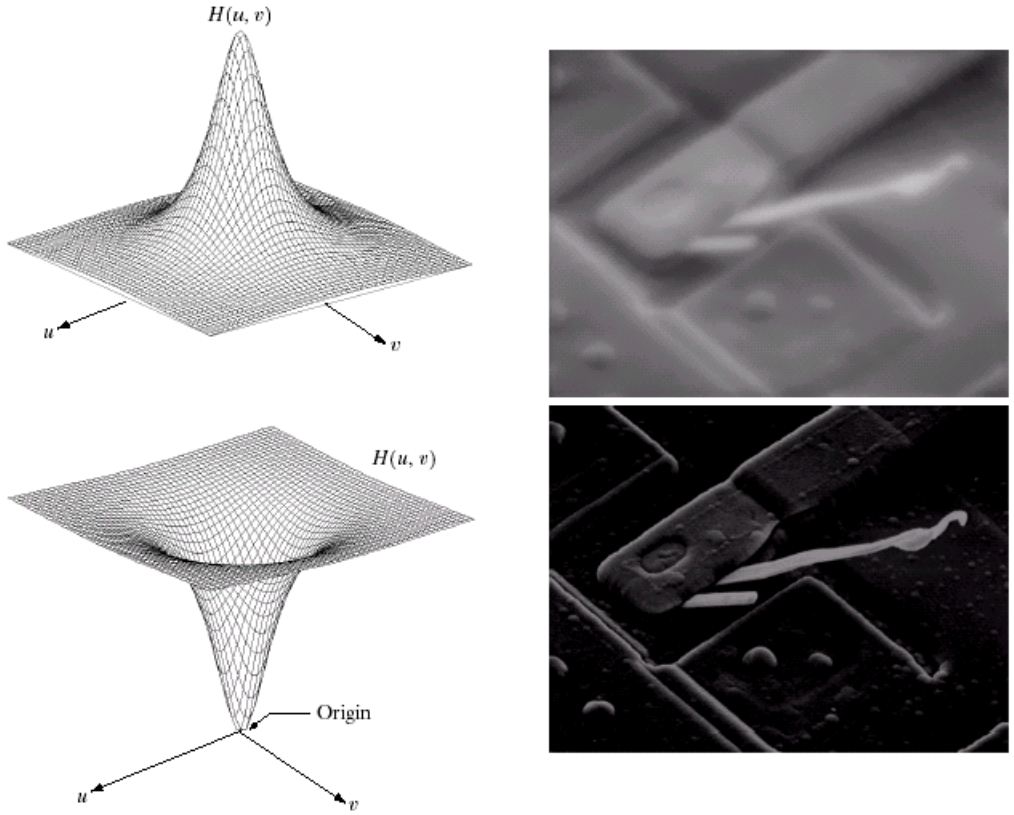


a
b

FIGURE 4.4 (a) SEM image of a damaged integrated circuit. (b) Fourier spectrum of (a). (Original image courtesy of Dr. J. M. Hudak, Brockhouse Institute for Materials Research, McMaster University, Hamilton, Ontario, Canada.)



Chapter 4
Image Enhancement in the Frequency Domain



a b
c d

FIGURE 4.7 (a) A two-dimensional lowpass filter function. (b) Result of lowpass filtering the image in Fig. 4.4(a). (c) A two-dimensional highpass filter function. (d) Result of highpass filtering the image in Fig. 4.4(a).

Reading

- Much more detailed slides
  by Steve Seitz, from UW CSE 576
  https://docs.google.com/presentation/d/1Yp0UiqIAUxsnGCyeEwfmYXMunxSNdqqMWtMSfyI8UUU/edit#slide=id.p
- Reading in Szeliski textbook:
  - 3.2 Linear Filtering, up to including 3.2.2
  - 3.5 Pyramids and Wavelets, up to including 3.5.3